

Глава 3. Специальный (ad-hoc) полиморфизм

ФП 2.0 · осень 2026

Андрей Стоян

осень 2026

План темы

- **Классы типов в языке:** словари, неявные аргументы, вывод инстансов, когерентность, дефункционализация
- **Семейства:** выбор по типу переносится с кода на данные и на сами типы
- **Кайнд *Constraint*:** ограничения как типы, с которыми можно вычислять
- **Использование:** что языки выражают через классы типов — от экзистенциальных типов до исключений

Где мы на карте

Что в этой главе	Откуда пришло и где выстрелит
словари классов типов	таблица свойств типа в Swift — глава 2; здесь набор операций задаёт программист
неявные аргументы	динамическое связывание — глава 5; системы эффектов — глава 8
дефункционализация	продолжения превращаются в стек фреймов — глава 6
семейства и <code>Constraint</code>	вычисления в типах из домашки 2; datatype-generic programming — глава 9
равенства типов, System FC	как внутри устроены GADT из главы 2
экзистенциальные типы, исключения	эффекты и хендлеры — главы 7–8

Перегрузка и классы типов

```
std::string toString(int x) { ... }           // перегрузка: одно имя, несколько функций
std::string toString(std::string fmt, double d) { ... }
```

```
class Show a where                          -- сначала объявляем, что перегружается и по какому параметру
  show :: a -> String

instance Show Int where
  show = ...
```

```
showPrefixed :: Show a => a -> String -> String      -- требуется словарь Show a
```

Выбор по типу, а не по значению

```
ghci> minBound :: Int           -- константа в классе: значения-получателя нет вовсе
-9223372036854775808

ghci> pure 1 :: Maybe Int       -- выбор по результату, и даже не по типу – по конструктору
Just 1

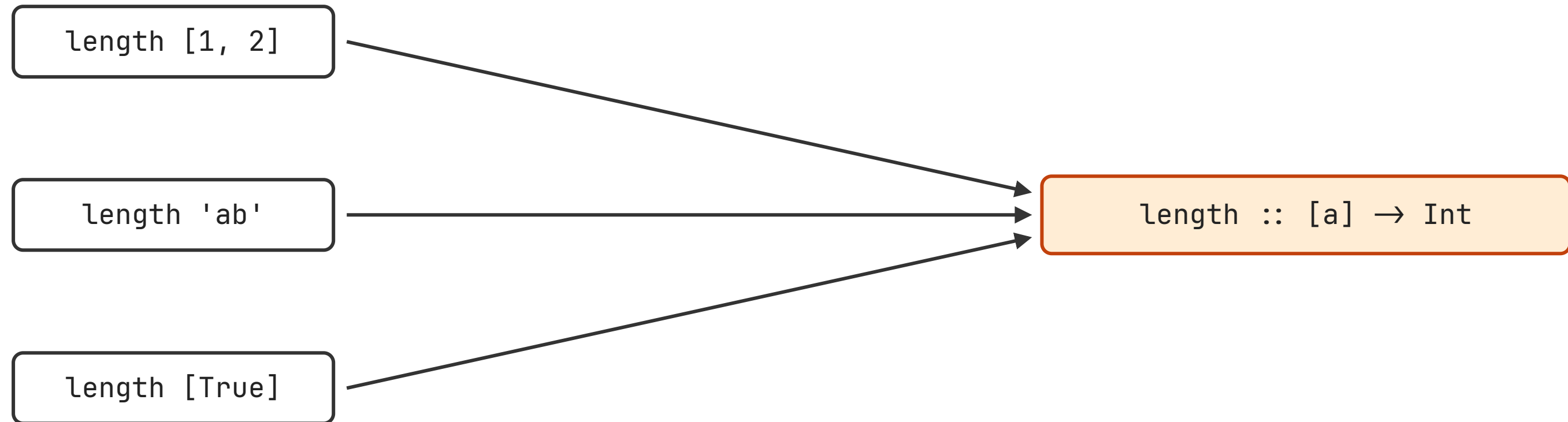
ghci> pure 1 :: [Int]
[1]
```

```
ghci> :k Show
Show :: Type -> Constraint      -- класс – не тип, а предикат на типах

ghci> :k [Show]
error: Expecting one more argument to ‘Show’
       Expected a type, but ‘Show’ has kind ‘Type -> Constraint’
```


Кто выбирает код

Параметрический полиморфизм: один и тот же код работает с данными любых типов.



Кто выбирает код

Специальный (ad-hoc) полиморфизм: имя одно, а код выбирается по типу.



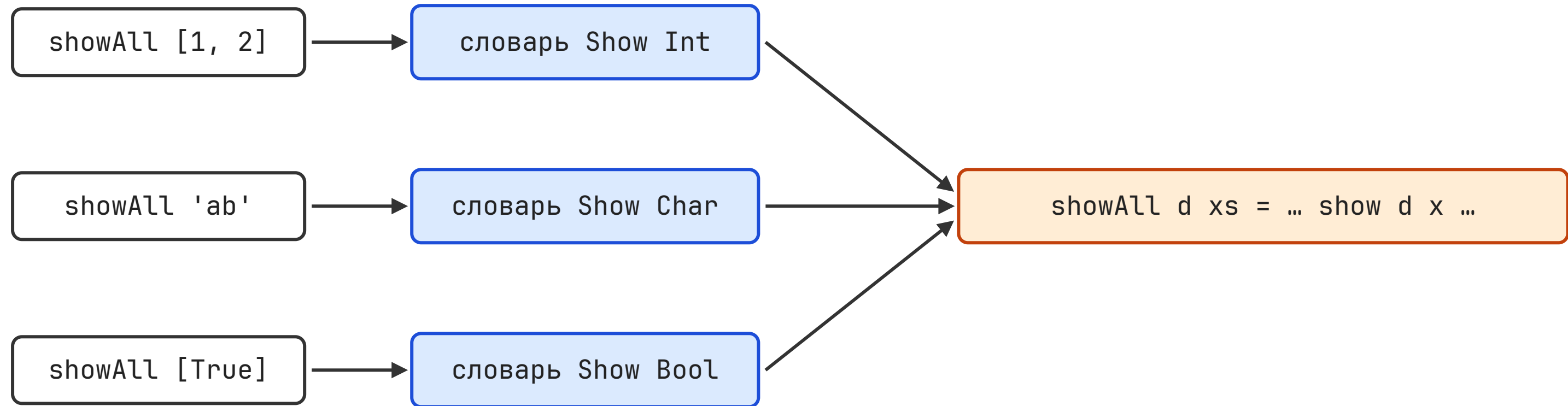
Кто выбирает код

А если вызов стоит внутри полиморфной функции? Тип `a` неизвестен — какой из трёх `show` вызывать?



Кто выбирает код

Код приезжает вместе с вызовом: скрытым аргументом передаётся таблица функций для типа. Это и есть класс типов.



Классы типов в языке

Что сортировка знает о типе

Сортировка вставками для чисел. От типа элементов ей нужно одно — порядок.

`sort :: [Int] → [Int]`

`insert x xs = ... partition (< x) xs ...`

Что сортировка знает о типе

Обобщаем по типу элементов. Параметрически-полиморфный код не знает про **a** ничего: сравнивать нечем.

`sort :: [a] → [a]`

`insert x xs = ... partition (? x) xs ...`

Что сортировка знает о типе

Значит, порядок придётся принять аргументом: словарь функций для типа **a**.

`sort :: OrdDict a → [a] → [a]`

`insert x xs = ... partition (`less` x) xs ...`

Словарь — обычное значение

```
data OrdDict a = OrdDict { less :: a -> a -> Bool }
```

```
intOrd :: OrdDict Int
```

```
intOrd = OrdDict { less = (<) }
```

```
ghci> sort intOrd [3, 2, 1]  
[1,2,3]
```

```
listOrd :: OrdDict a -> OrdDict [a]      -- порядок на списках — из порядка на элементах
```

```
listOrd d = OrdDict { less = ... less d ... }
```

```
ghci> sort (listOrd intOrd) [[3, 2], [2, 1], [0]]  
[[0],[2,1],[3,2]]
```

Задача: порядок на списках

✎ Допишите `listOrd` так, чтобы списки сравнивались лексикографически.

⚡ **Решение.**

```
listOrd :: OrdDict a -> OrdDict [a]
listOrd d = OrdDict { less = go }
  where
    go _ [] = False
    go [] _ = True
    go (x:xs) (y:ys) = less d x y || (not (less d y x) && go xs ys)
```

Компилятор подставляет словарь

Тот же код с классом типов. Словарей не видно.

`sort :: Ord a => [a] → [a]`

`sort [3, 2, 1]`

`sort [[3, 2], [2, 1], [0]]`

Компилятор подставляет словарь

Компилятор превращает $\Rightarrow$ в обычную стрелку, а в каждом месте вызова строит словарь по типу аргумента.

`sort :: OrdDict a → [a] → [a]`

`sort intOrd [3, 2, 1]`

`sort (listOrd intOrd) [[3, 2], [2, 1], [0]]`

Население типа словаря

Каждый инстанс даёт словарь или функцию, строящую словарь. Нужно населить `OrdDict [Int]`.

Головы инстансов

`instance Ord Int`

`instance Ord Bool`

`instance Ord a => Ord [a]`

Типы словарей и функций

`intOrd`

`boolOrd`

`listOrd`

`:: OrdDict Int`

`:: OrdDict Bool`

`:: ∀a. OrdDict a → OrdDict [a]`

Ожидаемый тип словаря

`OrdDict [Int]`

Терм, который строит компилятор

`□`

Население типа словаря

Смотрим под `OrdDict`: снаружи у `[Int]` конструктор `[]`. Подходит голова `Ord [a]`.

Головы инстансов

`instance Ord Int`

`instance Ord Bool`

`instance Ord a => Ord [a]`

Типы словарей и функций

`intOrd`

`boolOrd`

`listOrd`

`:: OrdDict Int`

`:: OrdDict Bool`

`:: ∀a. OrdDict a → OrdDict [a]`

Ожидаемый тип словаря

`OrdDict [Int]`

`[]`

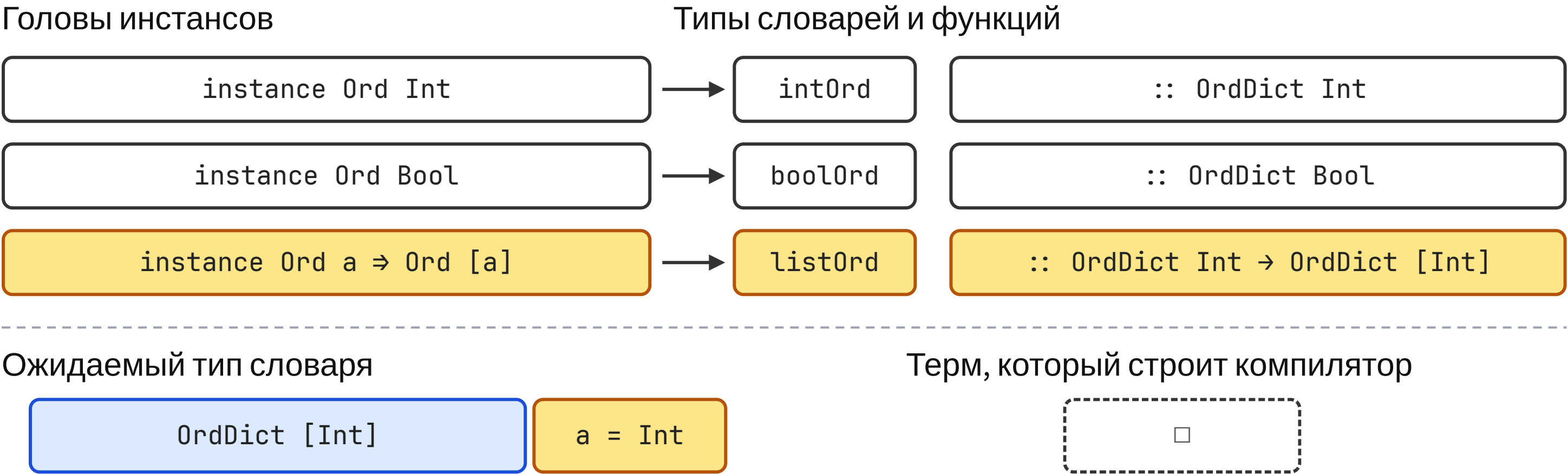
конструктор списка

Терм, который строит компилятор

`□`

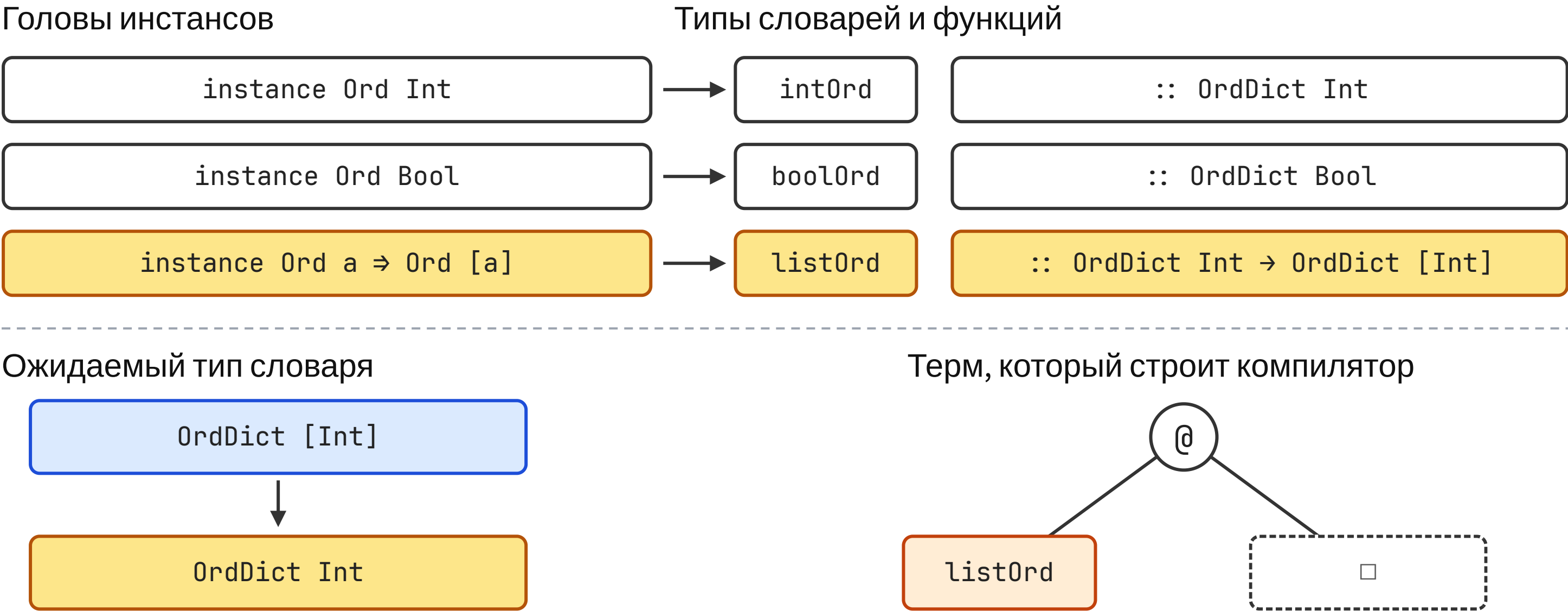
Население типа словаря

Сопоставление `[a]` с `[Int]` даёт `a = Int`. Подставляем его во весь тип `listOrd`.



Население типа словаря

Результат `listOrd` подходит. Его аргумент становится новой целью: `OrdDict Int`.



Население типа словаря

У новой цели снаружи `Int`. Подходит `Ord Int`: словарь `intOrd` не требует аргументов.

Головы инстансов

`instance Ord Int`

`instance Ord Bool`

`instance Ord a => Ord [a]`

Типы словарей и функций

`intOrd`

`boolOrd`

`listOrd`

`:: OrdDict Int`

`:: OrdDict Bool`

`:: OrdDict Int -> OrdDict [Int]`

Ожидаемый тип словаря

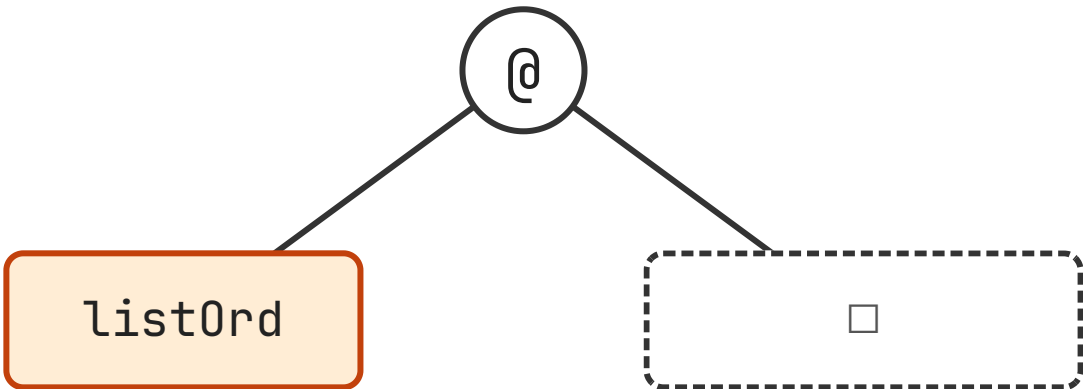
`OrdDict [Int]`



`OrdDict Int`

`Int`

Терм, который строит компилятор



Население типа словаря

Заполняем оставшуюся дырку: `intOrd :: OrdDict Int`. Терм словаря собран.

Головы инстансов

`instance Ord Int`

`instance Ord Bool`

`instance Ord a => Ord [a]`

Типы словарей и функций

`intOrd`

`boolOrd`

`listOrd`

`:: OrdDict Int`

`:: OrdDict Bool`

`:: OrdDict Int -> OrdDict [Int]`

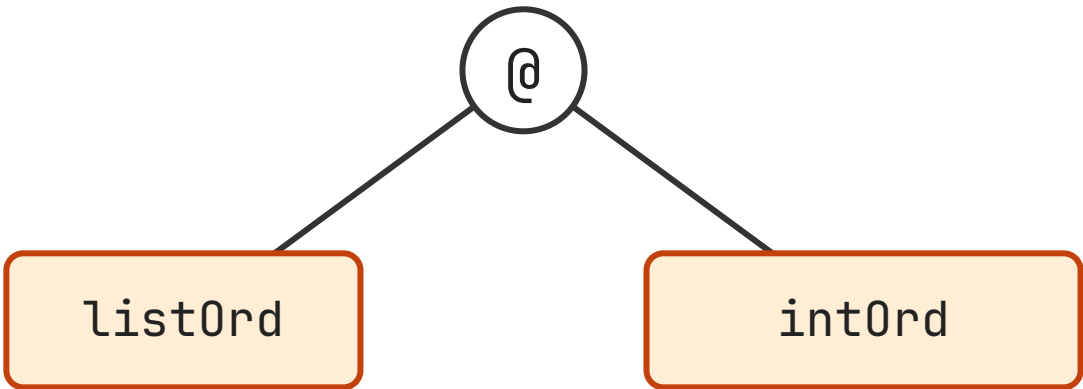
Ожидаемый тип словаря

`OrdDict [Int]`



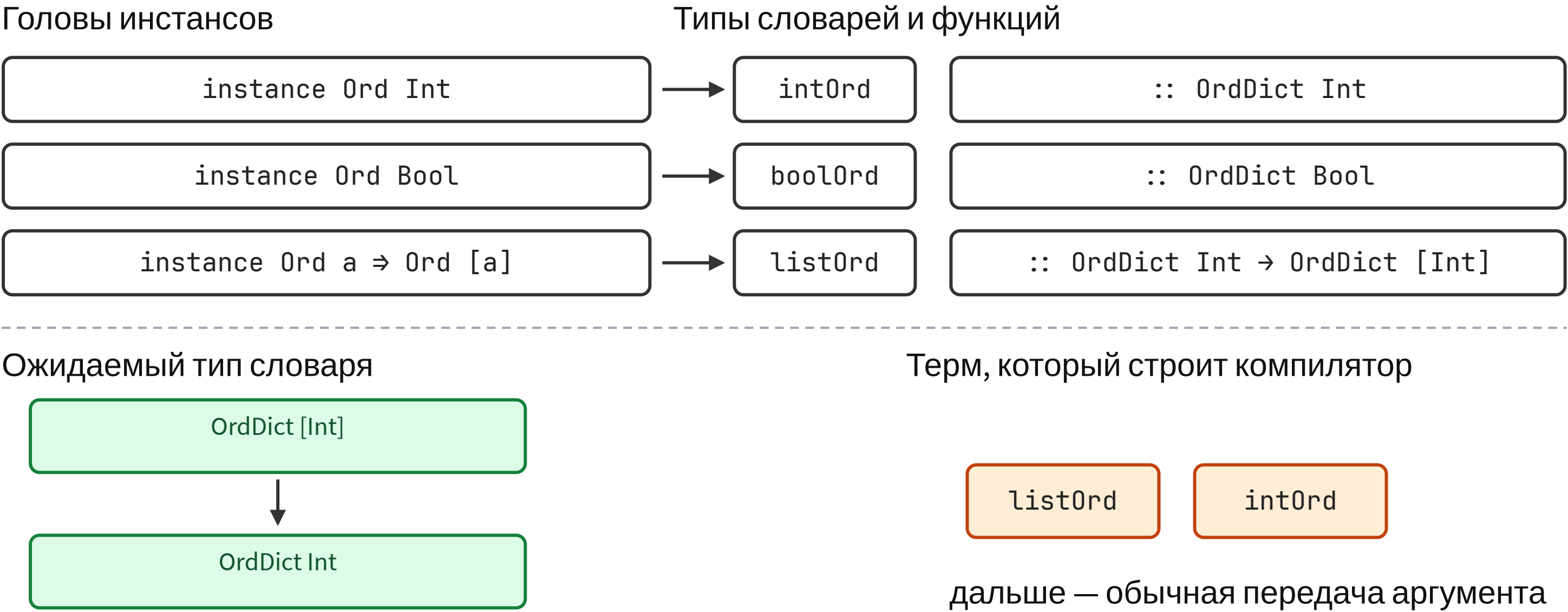
`OrdDict Int`

Терм, который строит компилятор



Население типа словаря

Получили `listOrd intOrd :: OrdDict [Int]`. Именно этот аргумент компилятор передаёт в `sort`.



Класс типов и словарь — одно и то же

Словарь: всё руками

```
-- 1. набор функций
data MyOrd a = MyOrd
  { less :: a -> a -> Bool }

-- 2. экземпляр: значение с именем
intMyOrd :: MyOrd Int
intMyOrd = MyOrd { less = (<) }

-- 3. явный параметр
sort :: MyOrd a -> [a] -> [a]

-- 4. передаёт программист
test = sort intMyOrd [3, 2, 1]
```

Класс типов: то же делает компилятор

```
-- 1. набор функций
class MyOrd a where
  less :: a -> a -> Bool

-- 2. экземпляр: имени нет
instance MyOrd Int where
  less = (<)

-- 3. неявный параметр
sort :: MyOrd a => [a] -> [a]

-- 4. передаёт компилятор
test = sort [3, 2, 1]
```

- **Δ Свидетель (witness)** — словарь как доказательство того, что тип удовлетворяет ограничению

Задача: словарь для Functor

✎ Какой словарь будет соответствовать higher-kinded классу типов Functor?

⚡ **Решение.**

```
data FunctorDict f = FunctorDict
  { fmap :: forall a b . (a -> b) -> f a -> f b }      -- поле — полиморфная функция: ранг 2

maybeFunctor :: FunctorDict Maybe
```

=> — стрелка с неявным аргументом

```
f :: (Show b => b) -> b
f x = x
```

```
error: No instance for 'Show b' arising from a use of 'x'
       Possible fix: add (Show b) to the context of the type signature for:
           f :: forall b. (Show b => b) -> b
```

```
f :: Show b => (Show b => b) -> b
f x = x
```

Словарь доходит до места использования

Аргумент x — функция от словаря. В теле f его надо к чему-то применить, а словаря в области видимости нет.

$$f :: (\text{ShowDict } b \rightarrow b) \rightarrow b$$
$$f\ x = x\ ?$$

Словарь доходит до места использования

Примем словарь аргументом f — и компилятор сам передаст его дальше.

$$f :: \text{ShowDict } b \rightarrow (\text{ShowDict } b \rightarrow b) \rightarrow b$$
$$f \, d \, x = x \, d$$

Неявным может быть любой аргумент

```
sort :: (?cmp :: a -> a -> Bool) => [a] -> [a]      -- ImplicitParams
sort = sortBy ?cmp

ghci> let ?cmp = (>) in sort [1, 3, 2]               -- берётся из области видимости по имени
[3,2,1]
```

```
data ShowDict a where
    ShowDict :: Show a => ShowDict a                -- словарь, сохранённый в структуре данных

f :: ShowDict b -> (Show b => b) -> b
f d x = case d of ShowDict -> x                    -- в ветке инстанс Show b доступен
```

Задача: когда приходит словарь

✎ **Устно:** Аргумент x имеет тип $\text{Show } b \Rightarrow b$, то есть ждёт словарь. В какой момент он его получает? Требуется ли такая передача словаря ленивости языка, или достаточно передать функцию от словаря? Что вычисляется до вызова f в энергичном языке?

⚡ **Решение.**

В месте использования, внутри ветки case : x применяется к словарю из обёртки. После рассахаривания x — функция от словаря. В энергичном языке до вызова достаточно получить её замыкание; тело выполнится при применении. Передача по имени не требуется.

Рекурсия по структуре типа

```
type data Nat = Zero | Suc Nat

class KnownNat (n :: Nat) where
  natVal :: Int                -- AllowAmbiguousTypes: n нет в типе метода

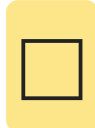
instance KnownNat Zero where
  natVal = 0

instance KnownNat n => KnownNat (Suc n) where
  natVal = 1 + natVal @n
```

```
ghci> natVal @(Suc (Suc Zero))
2
```


Как словарь превращается в число

`natVal` `@(Suc (Suc Zero))` превращается в вызов `natValD` со словарём. В словаре одно поле — число.

`natValD` 

```
data KnownNatDict (n :: Nat) = KN { natValD :: Int }

knownZero = KN { natValD = 0 }

knownSuc d = KN { natValD = 1 + natValD d }
```

Тип словаря

Терм словаря

```
KnownNatDict (Suc (Suc Zero))
```

Как словарь превращается в число

Компилятор выбирает инстанс для `Suc n`: вставляет `knownSuc` и ищет словарь для `n = Suc Zero`.

`natValD (knownSuc   )`

```
data KnownNatDict (n :: Nat) = KN { natValD :: Int }
```

```
knownZero = KN { natValD = 0 }
```

```
knownSuc d = KN { natValD = 1 + natValD d }
```

Тип словаря

Терм словаря

```
KnownNatDict (Suc (Suc Zero))
```



```
KnownNatDict (Suc Zero)
```

Как словарь превращается в число

Ещё один **Suc**: вставляем второй **knownSuc**. Остаётся найти словарь для **Zero**.

natValD (knownSuc (knownSuc □))

```
data KnownNatDict (n :: Nat) = KN { natValD :: Int }
```

```
knownZero = KN { natValD = 0 }
```

```
knownSuc d = KN { natValD = 1 + natValD d }
```

Тип словаря

Терм словаря

```
KnownNatDict (Suc (Suc Zero))
```



```
KnownNatDict (Suc Zero)
```

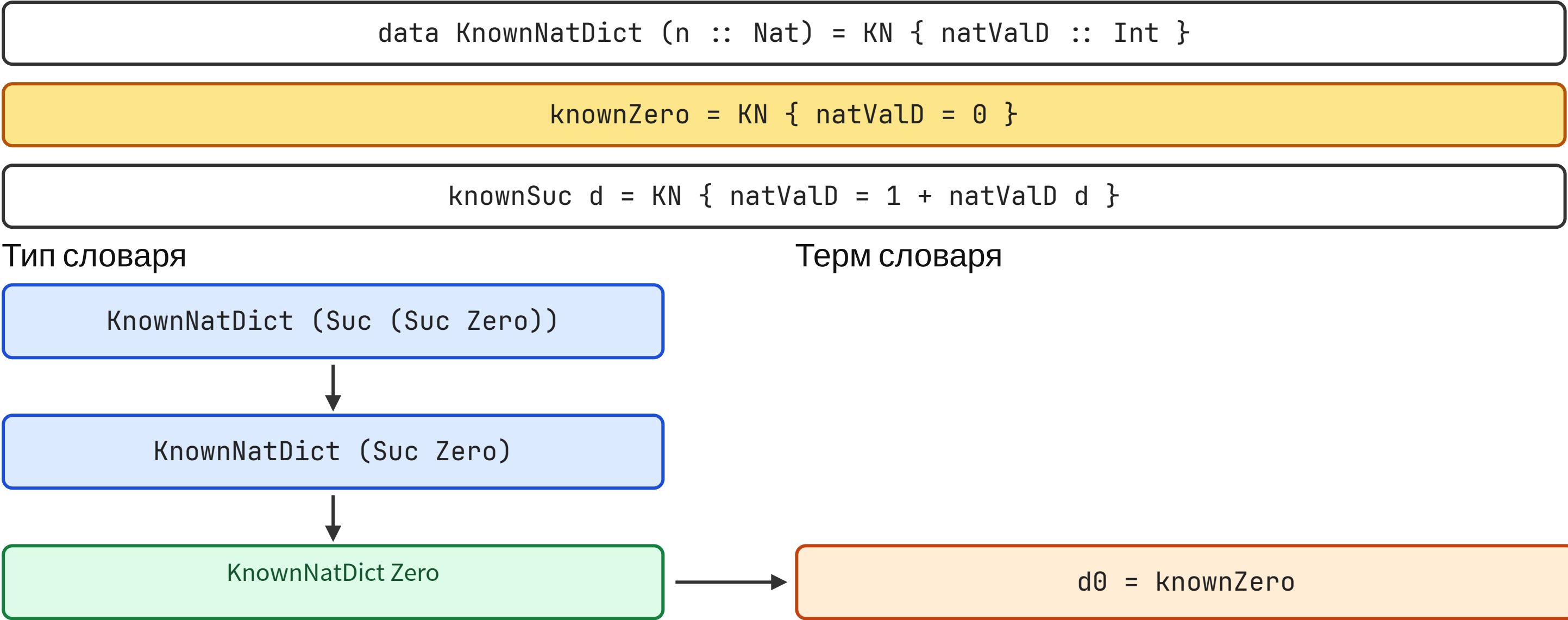


```
KnownNatDict Zero
```

Как словарь превращается в число

Для `Zero` есть готовый `knownZero`. Все дырки в терме заполнены; теперь собираем ответ снизу вверх.

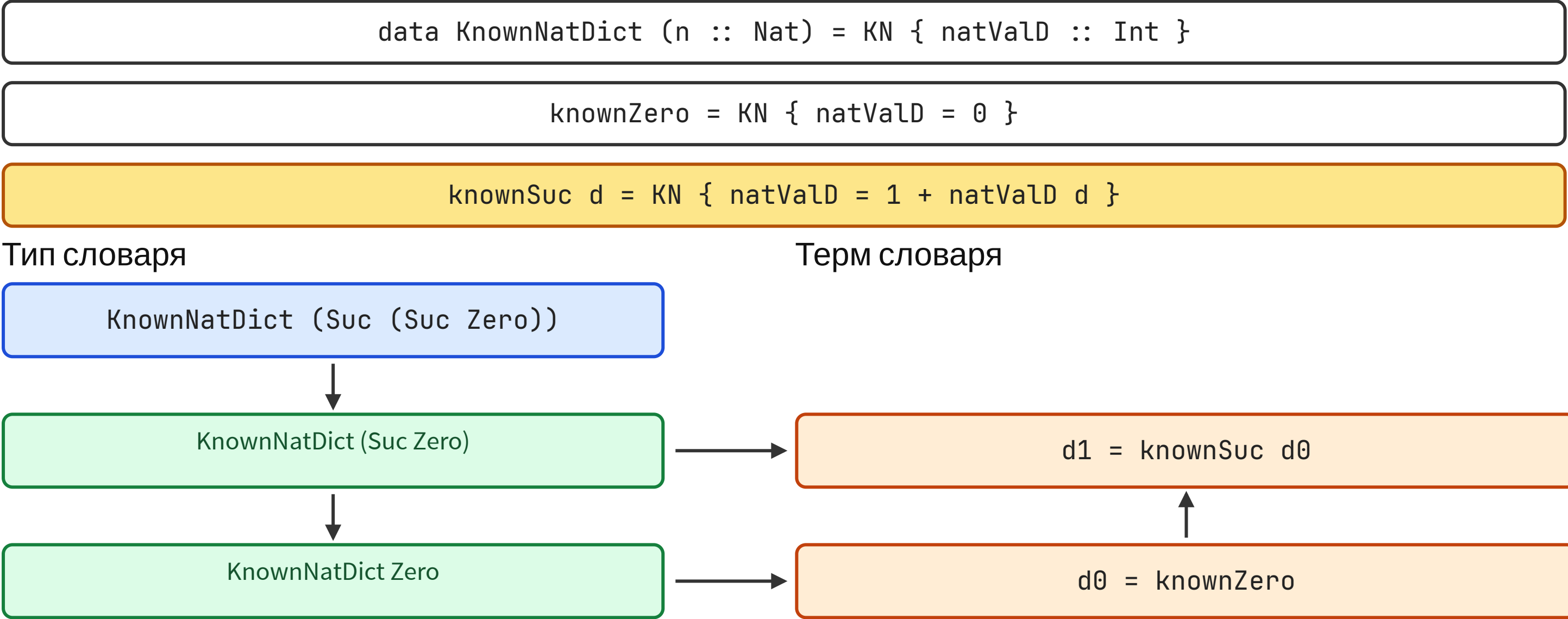
`natValD (knownSuc (knownSuc knownZero))`



Как словарь превращается в число

Внутренний словарь: $d1 = \text{knownSuc } d0$. Его поле задано выражением $1 + \text{natValD } d0$.

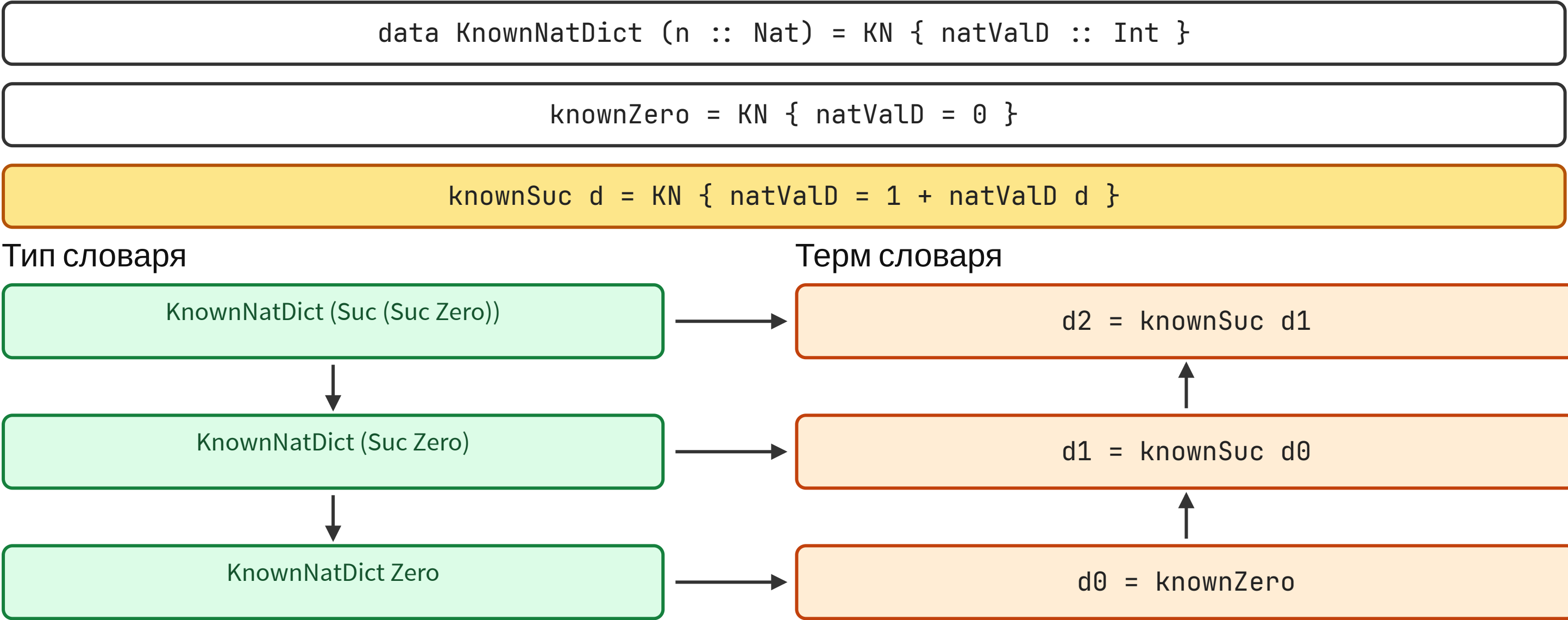
$\text{natValD } (\text{knownSuc } (\text{knownSuc } \text{knownZero}))$



Как словарь превращается в число

Внешний словарь: $d2 = \text{knownSuc } d1$. В вызов подставлен весь терм $\text{knownSuc } (\text{knownSuc } \text{knownZero})$.

$\text{natValD } (\text{knownSuc } (\text{knownSuc } \text{knownZero}))$



Как словарь превращается в число

Теперь выполнение: `natValD d2` выбирает поле внешнего словаря. Покажем содержимое каждого словаря.

`natValD d2`

```
data KnownNatDict (n :: Nat) = KN { natValD :: Int }

knownZero = KN { natValD = 0 }

knownSuc d = KN { natValD = 1 + natValD d }
```

Тип словаря

- KnownNatDict (Suc (Suc Zero))
- KnownNatDict (Suc Zero)
- KnownNatDict Zero

Словарь и его поле

- `d2 = KN { natValD = 1 + natValD d1 }`
- `d1 = KN { natValD = 1 + natValD d0 }`
- `d0 = KN { natValD = 0 }`

Как словарь превращается в число

Поле `d2` — это `1 + natValD d1`: одна единица и чтение поля следующего словаря.

`1 + natValD d1`

```
data KnownNatDict (n :: Nat) = KN { natValD :: Int }

knownZero = KN { natValD = 0 }

knownSuc d = KN { natValD = 1 + natValD d }
```

Тип словаря

- KnownNatDict (Suc (Suc Zero))
- KnownNatDict (Suc Zero)
- KnownNatDict Zero

Словарь и его поле

- `d2 = KN { natValD = 1 + natValD d1 }`
- `d1 = KN { natValD = 1 + natValD d0 }`
- `d0 = KN { natValD = 0 }`

Как словарь превращается в число

Поле `d1` — ещё одна единица и `natValD d0`. Обе единицы получены из тела `knownSuc`.

$1 + (1 + \text{natValD } d0)$

```
data KnownNatDict (n :: Nat) = KN { natValD :: Int }  
  
knownZero = KN { natValD = 0 }  
  
knownSuc d = KN { natValD = 1 + natValD d }
```

Тип словаря

- KnownNatDict (Suc (Suc Zero))
- KnownNatDict (Suc Zero)
- KnownNatDict Zero

Словарь и его поле

- d2 = KN { natValD = 1 + natValD d1 }
- d1 = KN { natValD = 1 + natValD d0 }
- d0 = KN { natValD = 0 }

Как словарь превращается в число

Поле `d0` равно `0`: это тело `knownZero`. Осталось выполнить два сложения.

$1 + (1 + 0)$

```
data KnownNatDict (n :: Nat) = KN { natValD :: Int }  
  
knownZero = KN { natValD = 0 }  
  
knownSuc d = KN { natValD = 1 + natValD d }
```

Тип словаря

- KnownNatDict (Suc (Suc Zero))
- KnownNatDict (Suc Zero)
- KnownNatDict Zero

Словарь и его поле

- d2 = KN { natValD = 1 + natValD d1 }
- d1 = KN { natValD = 1 + natValD d0 }
- d0 = KN { natValD = 0 }

Как словарь превращается в число

Внутреннее сложение даёт 1. Это значение поля `natValD` у словаря `d1`.

1 + 1

data KnownNatDict (n :: Nat) = KN { natValD :: Int }

knownZero = KN { natValD = 0 }

knownSuc d = KN { natValD = 1 + natValD d }

Тип словаря

- KnownNatDict (Suc (Suc Zero))
- KnownNatDict (Suc Zero)
- KnownNatDict Zero

Словарь и его поле

- d2 = KN { natValD = 1 + natValD d1 }
- d1 = KN { natValD = 1 }
- d0 = KN { natValD = 0 }

Как словарь превращается в число

Внешнее сложение даёт 2. Это значение поля `natValD` у словаря `d2` — и ответ исходного вызова.

2

```
data KnownNatDict (n :: Nat) = KN { natValD :: Int }

knownZero = KN { natValD = 0 }

knownSuc d = KN { natValD = 1 + natValD d }
```

Тип словаря

- KnownNatDict (Suc (Suc Zero))
- KnownNatDict (Suc Zero)
- KnownNatDict Zero

Словарь и его поле

- d2 = KN { natValD = 2 }
- d1 = KN { natValD = 1 }
- d0 = KN { natValD = 0 }

Инстансы как программа

```
class Reverse (acc :: [Type]) (tys :: [Type]) where
  showReverse :: String

instance ShowT acc => Reverse acc '[] where
  showReverse = showTypes @acc

instance Reverse (ty : acc) tys => Reverse acc (ty : tys) where
  showReverse = showReverse @(ty : acc) @tys
```

```
ghci> showReverse @'[] @[Char, Int, Double]
"Double Int Char"
```


Разворот списка типов по шагам

Каждый шаг вывода — выбор инстанса по голове. Подошёл второй: первый тип переезжает в аккумулятор.

Reverse '[]' [**Char**, Int, Double]

Разворот списка типов по шагам

Посылка второго инстанса — снова `Reverse`, с аккумулятором на один тип длиннее.

`Reverse '[Char] '[Int, Double]`

Разворот списка типов по шагам

Ещё шаг.

Reverse '[Int, Char] '[Double]

Разворот списка типов по шагам

Список опустел — теперь подходит первый инстанс.

Reverse '[Double, Int, Char] '[]

Разворот списка типов по шагам

Его посылка — `ShowT` для аккумулятора. При компиляции построен терм со словарями; при исполнении его методы строят строку.

`ShowT '[Double, Int, Char]`

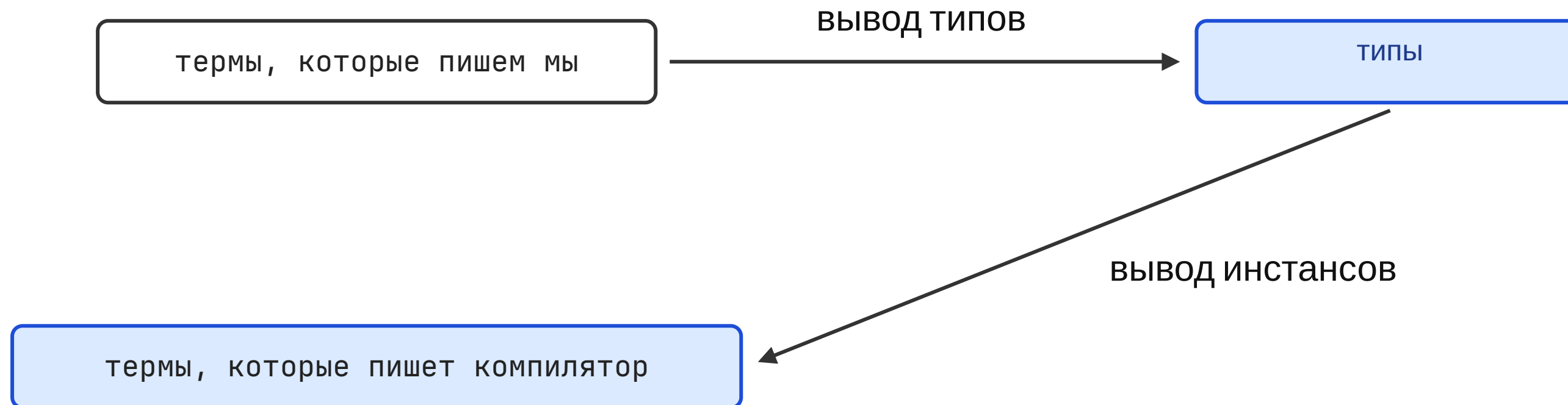
Классы типов и вывод инстансов — кодогенерация

Мы пишем термы; по ним компилятор выводит типы.



Классы типов и вывод инстансов — кодогенерация

А по типам — инстансы, то есть снова термы. Интересный код пишем мы, неинтересный параллельно выписывает компилятор.



Из значения — в тип

```
reify :: Int -> (forall n. KnownNat n => a) -> a      -- тип вернуть нельзя: принимаем продолжение
reify n k
  | n <= 0 = k @Zero                                -- передаётся knownZero
  | otherwise = reify (n - 1) \ @n' -> k @(Suc n')    -- передаётся knownSuc d
                                                    -- d — словарь KnownNat n'
```

```
wonderId :: Int -> Int
wonderId n = reify n (\ @t -> natVal @t)              -- значение → тип → значение
```


reify 2: число убывает, словарь растёт

Покажем словарный перевод `reify` как `reifyD`: продолжение получает типовой аргумент и явный словарь.

`reifyD` 2 `k`

```
k :: ∀n. KnownNatDict n → a
```

Тип словаря

Словарь, передаваемый продолжению

reify 2: число убывает, словарь растёт

Число уменьшается до 1. Новое продолжение k_1 получит словарь d и передаст в k словарь `knownSuc d`.

reifyD 1 k_1

$k :: \forall n. \text{KnownNatDict } n \rightarrow a$

$k_1 = \lambda n. \lambda d. k \text{ @(Suc } n) (\text{knownSuc } d)$

Тип словаря

Словарь, передаваемый продолжению

reify 2: число убывает, словарь растёт

Число уменьшается до 0. Продолжение k_2 добавит ещё один `knownSuc`, когда получит свой словарь.

reifyD 0 k_2

$k :: \forall n. \text{KnownNatDict } n \rightarrow a$

$k_1 = \lambda n. \lambda d. k \text{ @(Suc } n) (\text{knownSuc } d)$

$k_2 = \lambda n. \lambda d. k_1 \text{ @(Suc } n) (\text{knownSuc } d)$

Тип словаря

Словарь, передаваемый продолжению

reify 2: число убывает, словарь растёт

База рекурсии: выбираем `Zero`, строим `d0 = knownZero` и передаём его продолжению `k2`.

`k2` `@Zero` `d0`

`k :: ∀n. KnownNatDict n → a`

`k1 = λn. λd. k @(Suc n) (knownSuc d)`

`k2 = λn. λd. k1 @(Suc n) (knownSuc d)`

Тип словаря

`KnownNatDict Zero`

Словарь, передаваемый продолжению

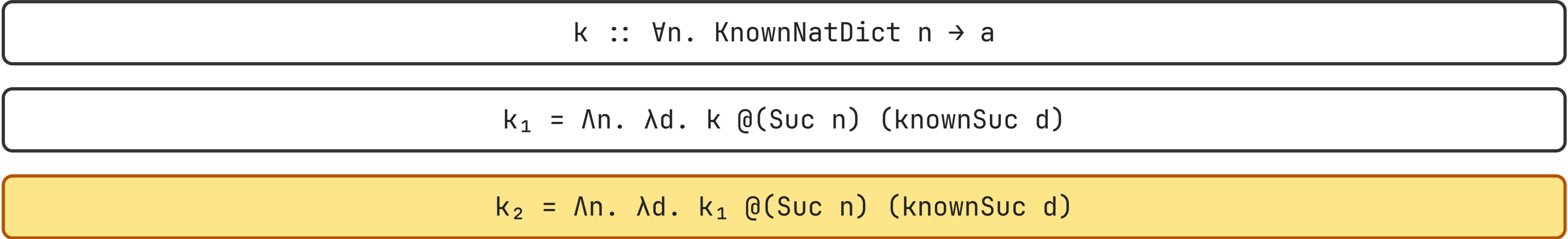
`d0 = knownZero`



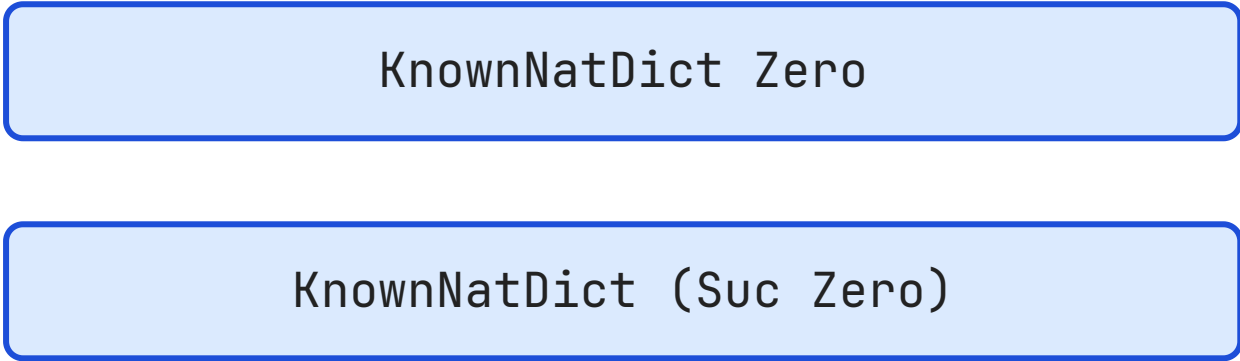
reify 2: число убывает, словарь растёт

Вызываем k_2 : из $d0$ строится $d1 = \text{knownSuc } d0$. Продолжение k_1 получает тип Suc Zero и этот словарь.

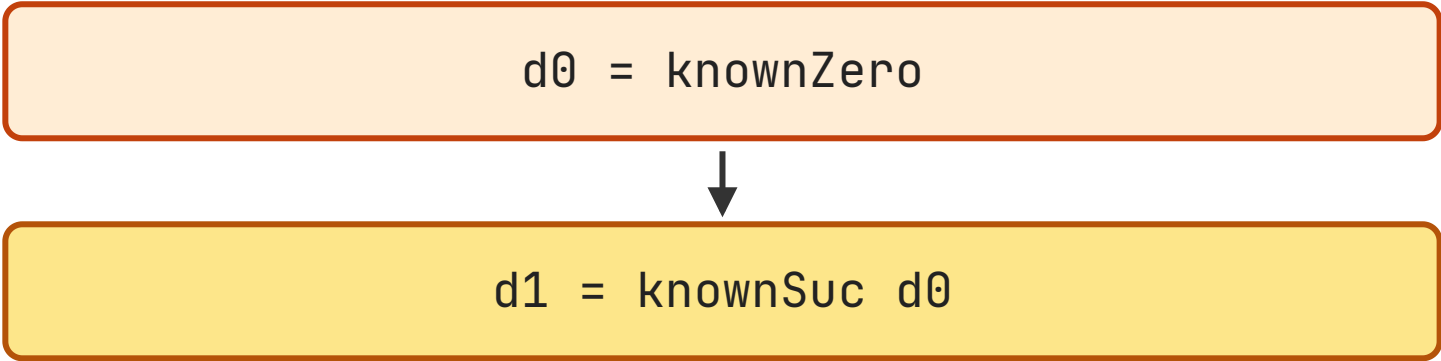
k_1 @(Suc Zero) $d1$



Тип словаря



Словарь, передаваемый продолжению



reify 2: число убывает, словарь растёт

Вызываем k_1 : строится $d2 = \text{knownSuc } d1$. Исходное k получает тип $\text{Suc } (\text{Suc } \text{Zero})$ и готовый словарь.

k $@(\text{Suc } (\text{Suc } \text{Zero}))$ $d2$

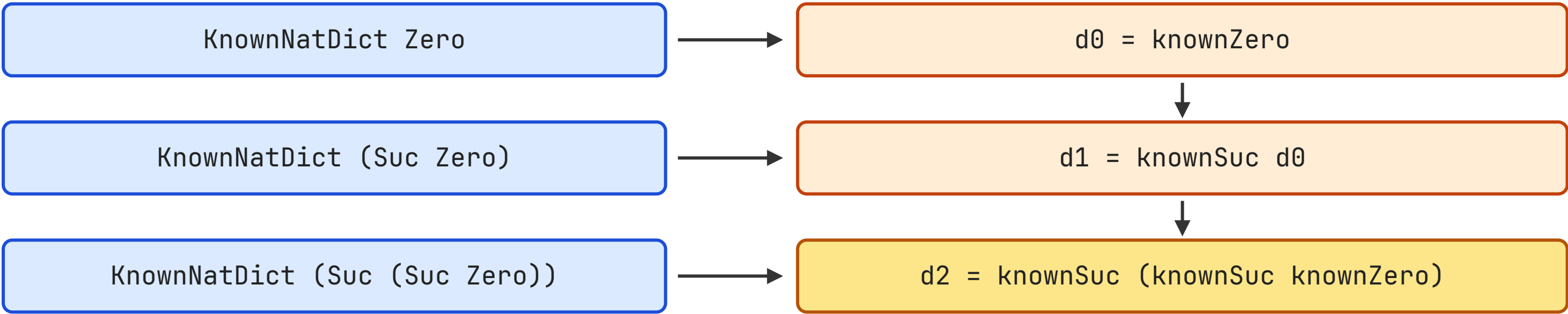
$k :: \forall n. \text{KnownNatDict } n \rightarrow a$

$k_1 = \lambda n. \lambda d. k @(\text{Suc } n) (\text{knownSuc } d)$

$k_2 = \lambda n. \lambda d. k_1 @(\text{Suc } n) (\text{knownSuc } d)$

Тип словаря

Словарь, передаваемый продолжению



Задача: удивительная тождественная функция

✎ **Устно:** Что вернёт `wonderId (-3)`? Является ли `wonderId` тождественной функцией?

⚡ **Решение.**

```
ghci> wonderId 5
5

ghci> wonderId (-3)
0
```

Для $n \leq 0$ выбирается тип `Zero`: функция тождественна только на неотрицательных числах.

Классы типов из имплицитов: Scala

```
trait Show[T] { def show(x: T): String }           // класс

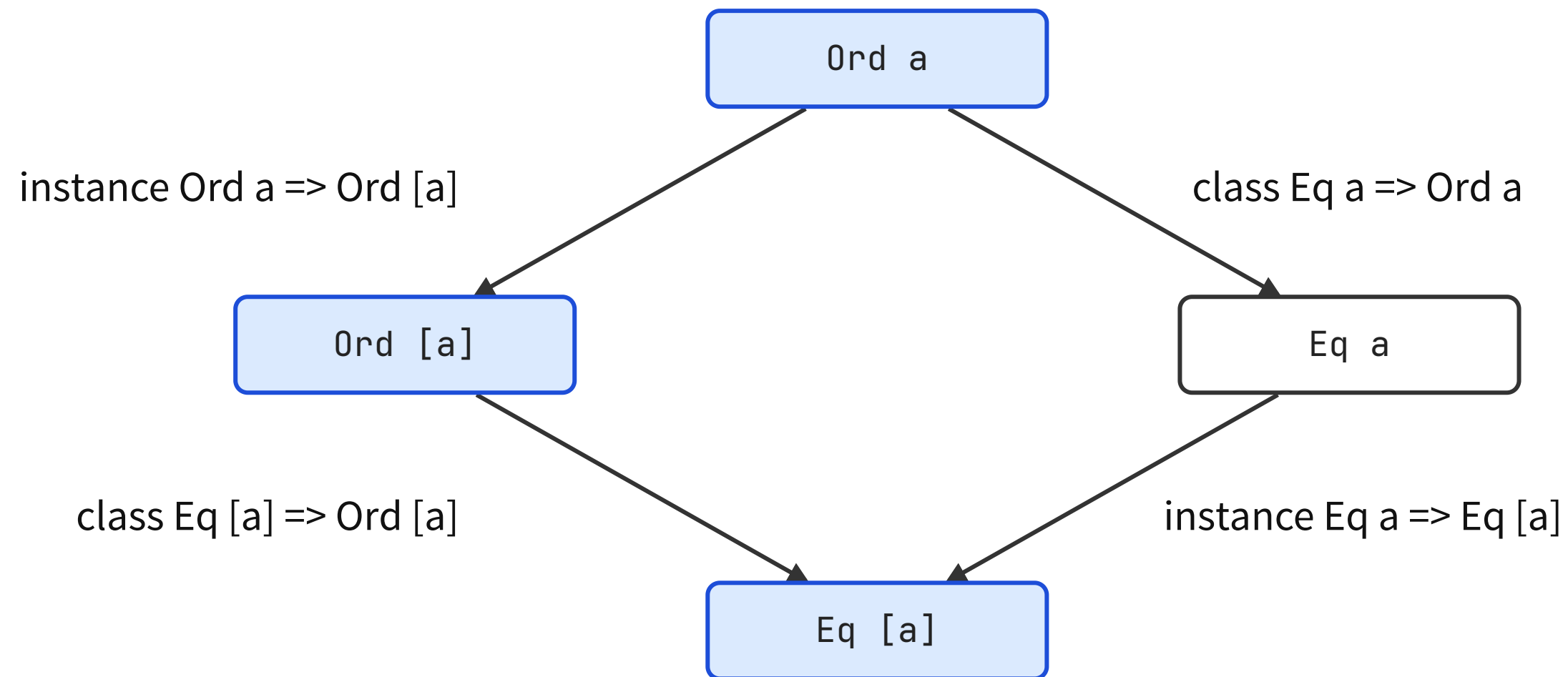
implicit object intShow extends Show[Int] {         // словарь
  def show(x: Int): String = x.toString
}

def showAll[T](xs: List[T])(implicit ev: Show[T]): String = // параметр
  xs.map(ev.show(_)).mkString(", ")
```

```
showAll(List(1, 2, 3))           // "1, 2, 3": компилятор подставляет intShow
showAll(List(1, 2, 3))(intShow) // "1, 2, 3": передаём словарь сами
implicitly[Show[Int]].show(42)   // "42": получаем словарь и вызываем его метод
```

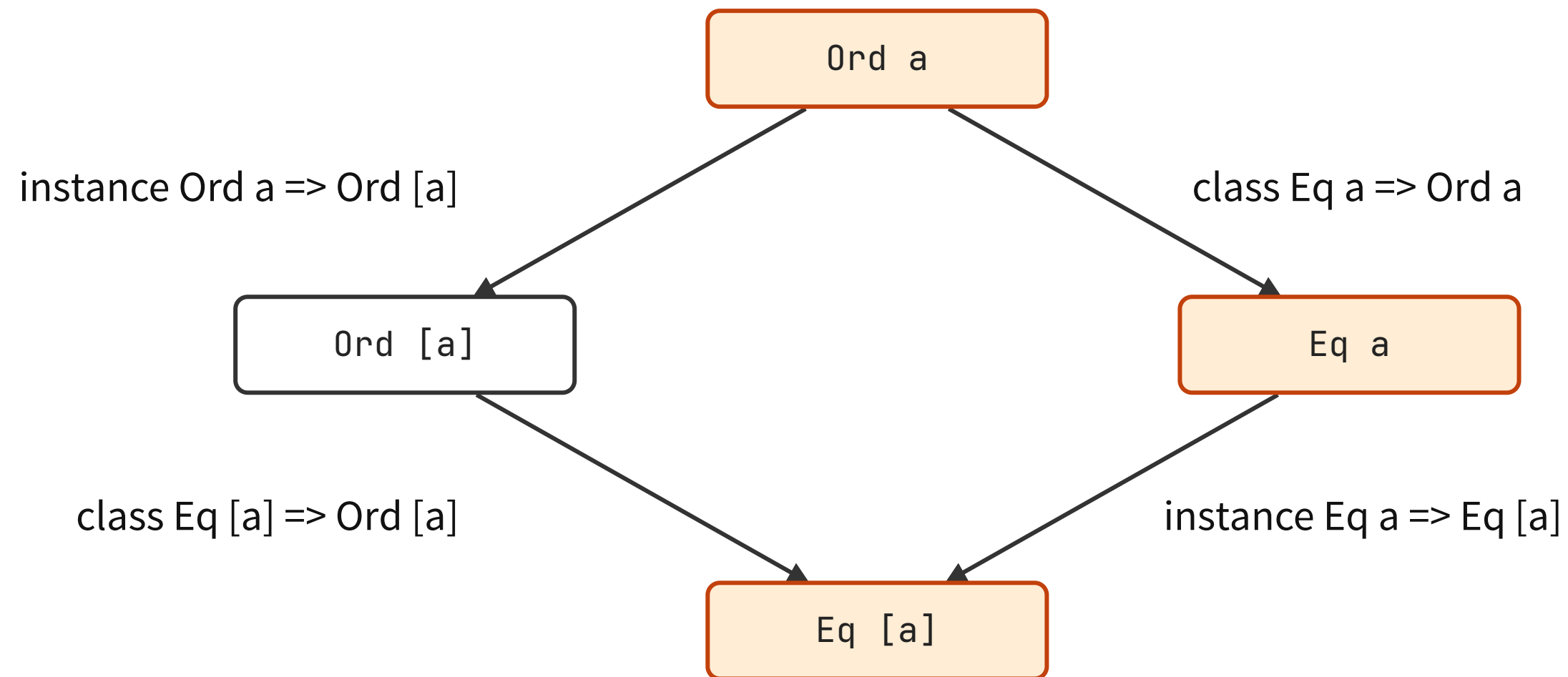

Когерентность: два пути — один словарь

Нужен словарь `Eq [a]`, в контексте есть `Ord a`. Первый путь: порядок на списках, а из него равенство.



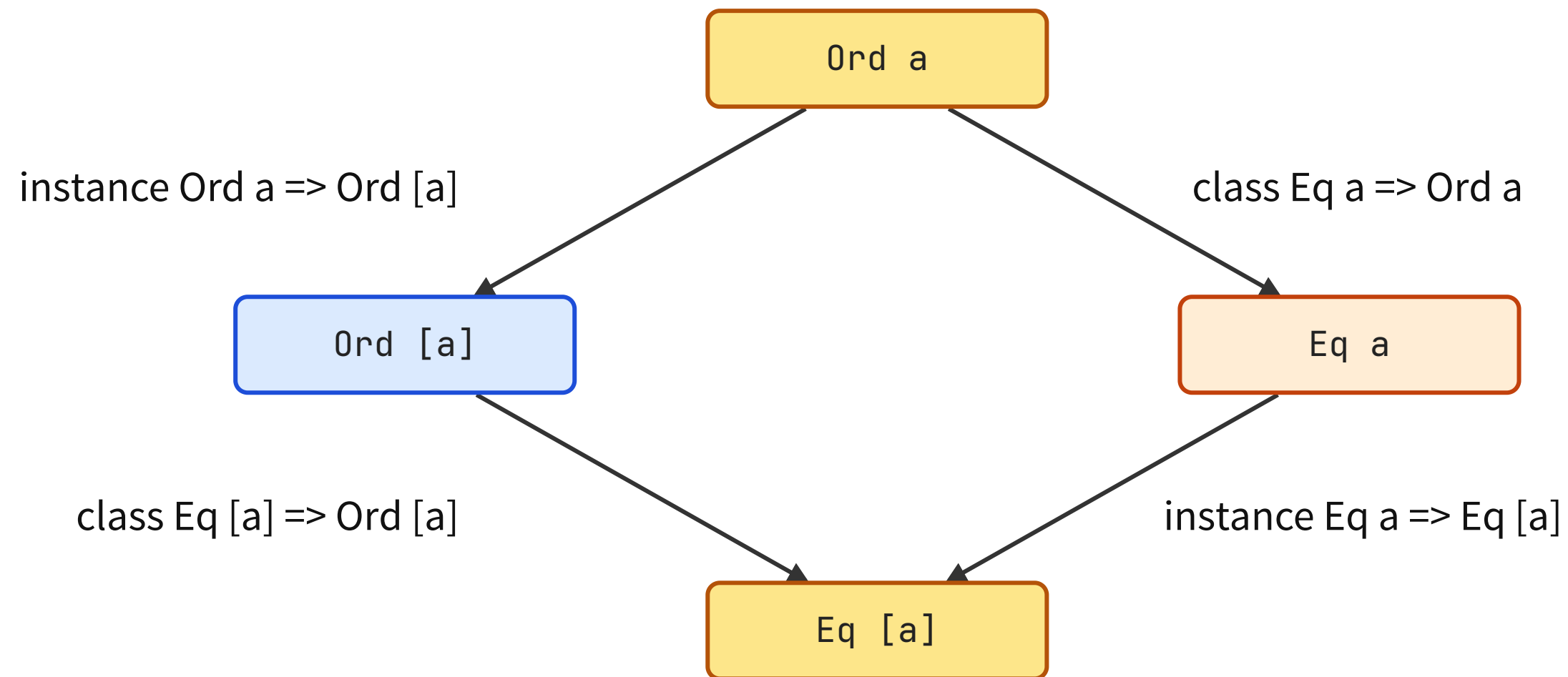
Когерентность: два пути — один словарь

Второй путь: равенство на элементах, а из него равенство на списках.



Когерентность: два пути — один словарь

Когерентность: словари, полученные разными путями, неотличимы — диаграмма коммутирует. Откуда пришёл инстанс, можно не думать.



Задача: множество с двумя порядками

✎ **Устно:** Множество `Set a` хранит элементы упорядоченными и при вставке и поиске пользуется порядком `Ord a`. Что произойдёт, если множество построить с одним порядком на элементах, а искать в нём с другим? Почему обычная когерентная модель классов типов в Haskell исключает такую ситуацию, а выбор импlicitа из локального контекста допускает её?

⚡ **Решение.**

Поиск свернёт не в то поддерево: элемент в множестве есть, а ответ — «нет». В обычной когерентной модели Haskell `insert` и `member` получают неотличимые словари `Ord a`. Импlicit берётся из области видимости места вызова — в двух местах реализации могут оказаться разными.

Правила переписывания и специализация

```
{-# RULES
  "map/map"      forall f g xs. map f (map g xs) = map (f . g) xs      -- закон функтора
  "map/append"    forall f xs ys. map f (xs ++ ys) = map f xs ++ map f ys
#-}
```

```
genericLookup :: Ord a => Table a b -> a -> b
intLookup     ::          Table Int b -> Int -> b

{-# RULES "genericLookup/Int" genericLookup = intLookup #-}
```

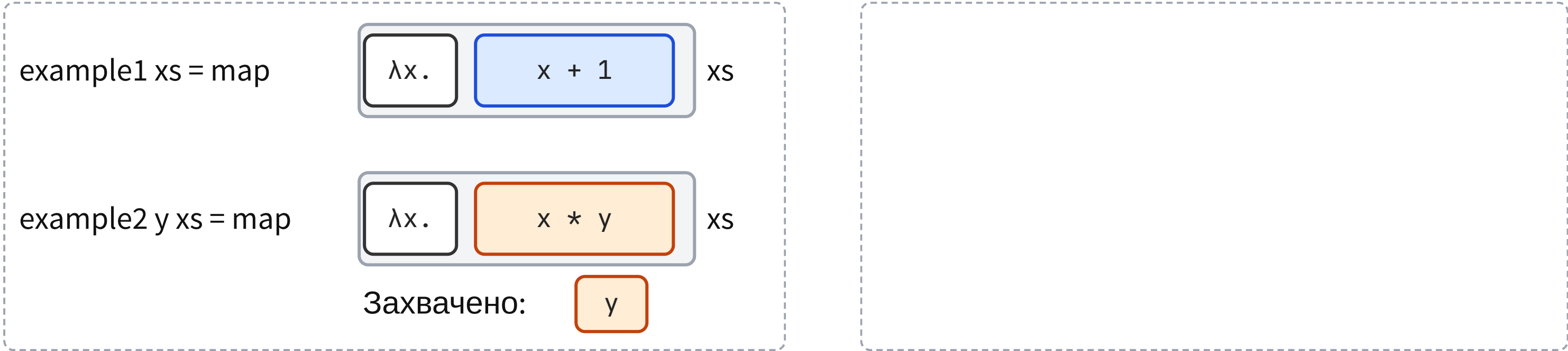
Дефункционализация: лямбды становятся данными

У каждой лямбды есть тело; вторая ещё захватывает y из места вызова. Найдём коду и данным новое место.

$\text{map} :: (\text{Int} \rightarrow \text{Int}) \rightarrow [\text{Int}] \rightarrow [\text{Int}]$

Места вызова: здесь остаются данные

apply: сюда переезжает код



data Fun = ...

Вызов внутри map

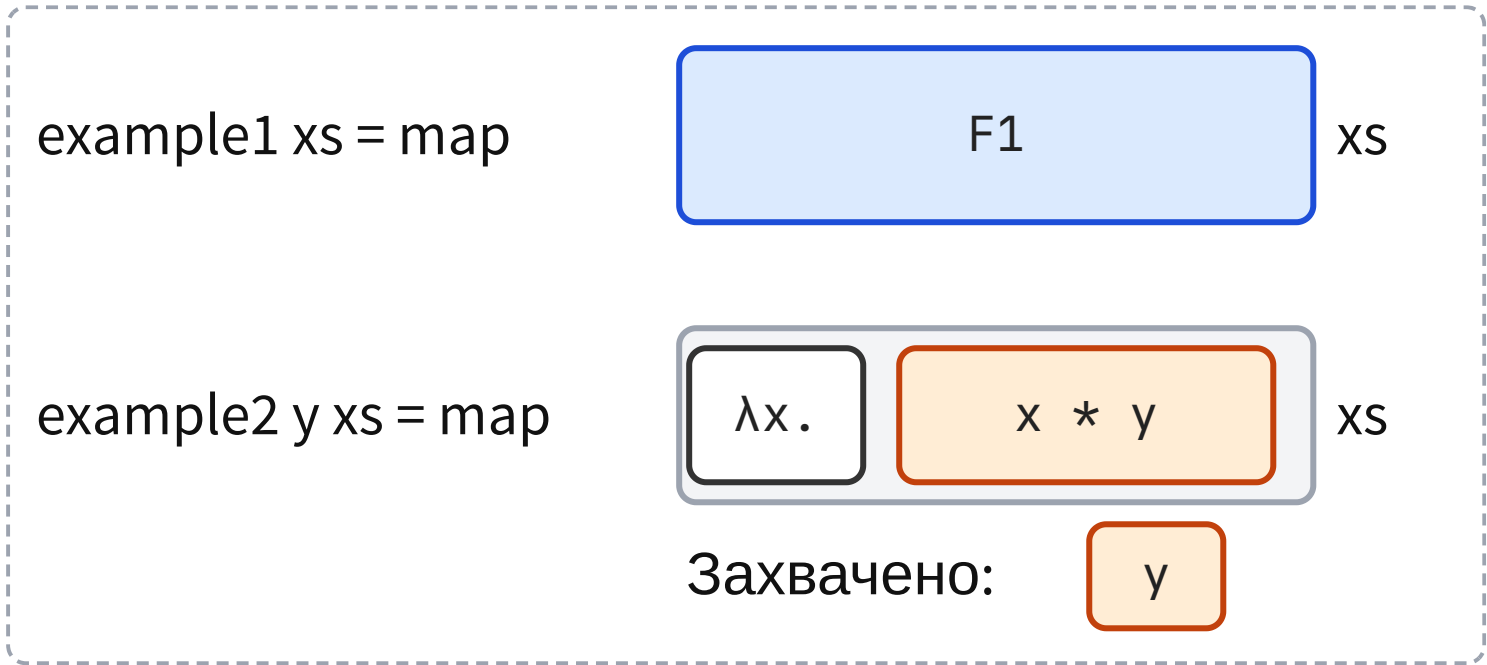


Дефункционализация: лямбды становятся данными

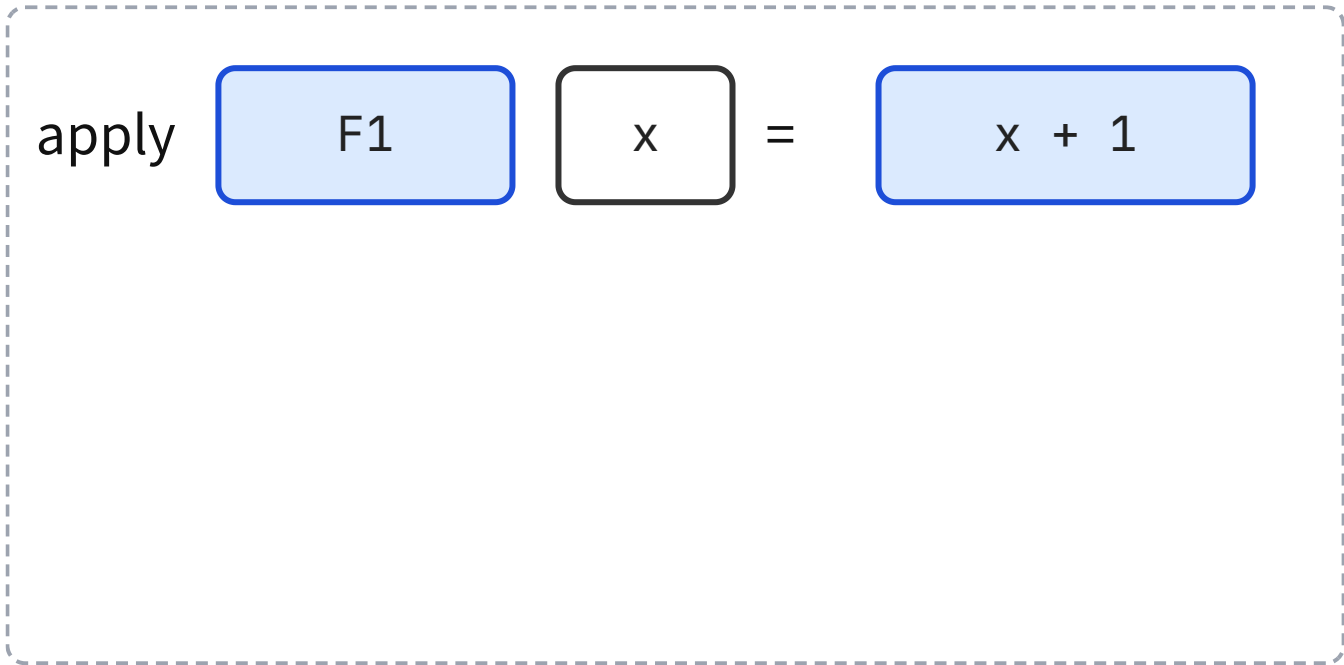
Тело $x + 1$ переезжает в ветку `apply F1 x`. На месте первой лямбды остаётся конструктор `F1`.

`map :: (Int → Int) → [Int] → [Int]`

Места вызова: здесь остаются данные

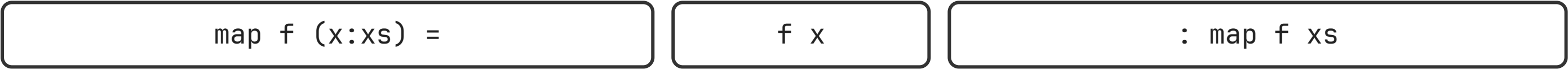


apply: сюда переезжает код



data Fun = F1 | ...

Вызов внутри map

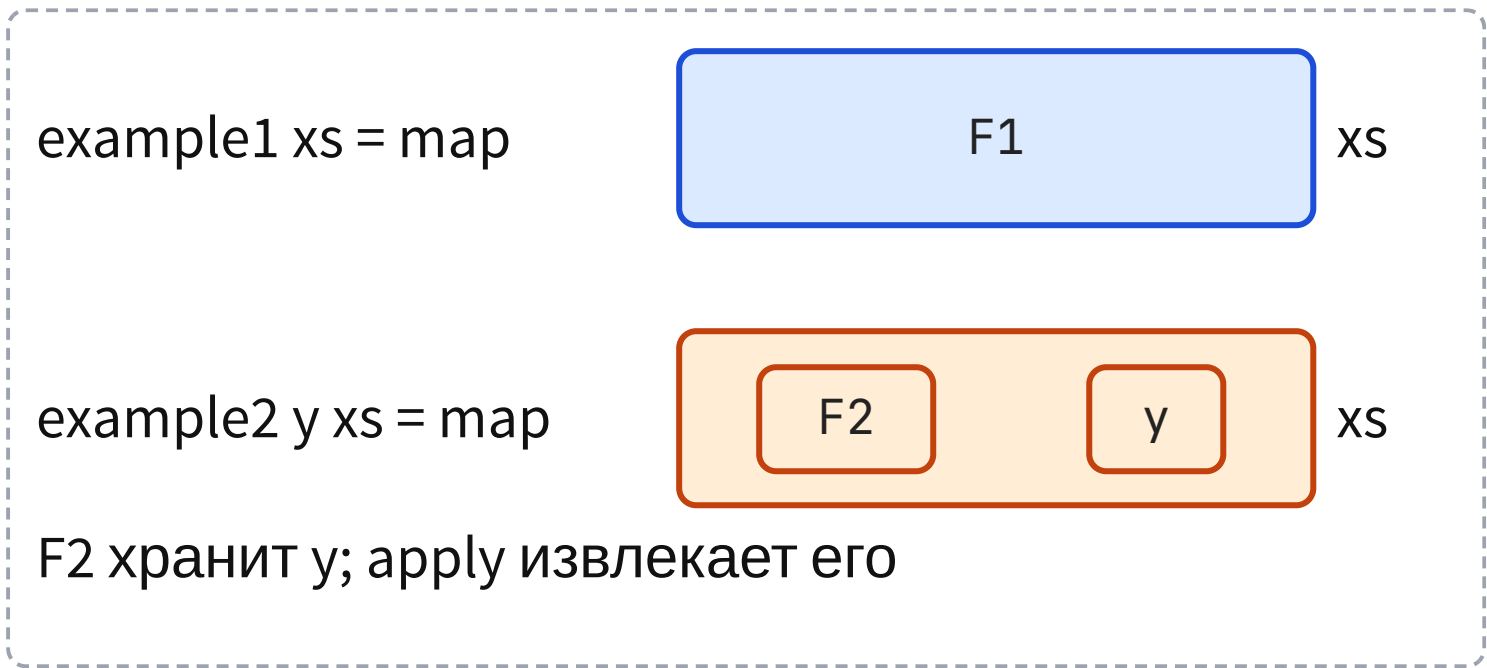


Дефункционализация: лямбды становятся данными

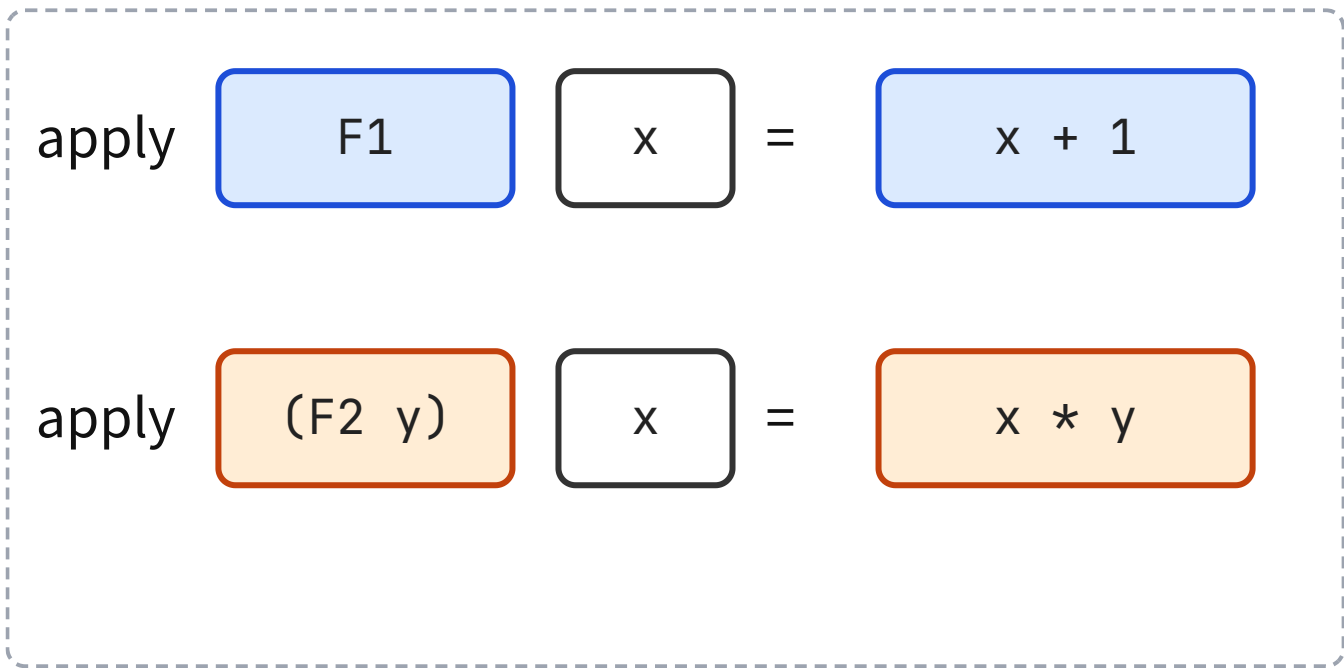
Тело $x * y$ переезжает в ветку `apply (F2 y) x`. Захваченное y становится полем конструктора `F2`.

`map :: (Int → Int) → [Int] → [Int]`

Места вызова: здесь остаются данные

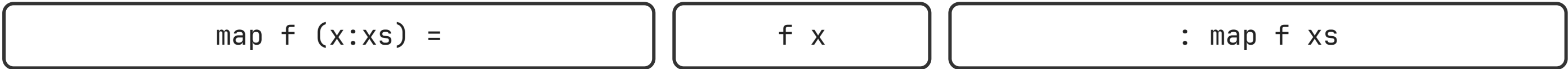


apply: сюда переезжает код



`data Fun = F1 | F2 Int`

Вызов внутри map

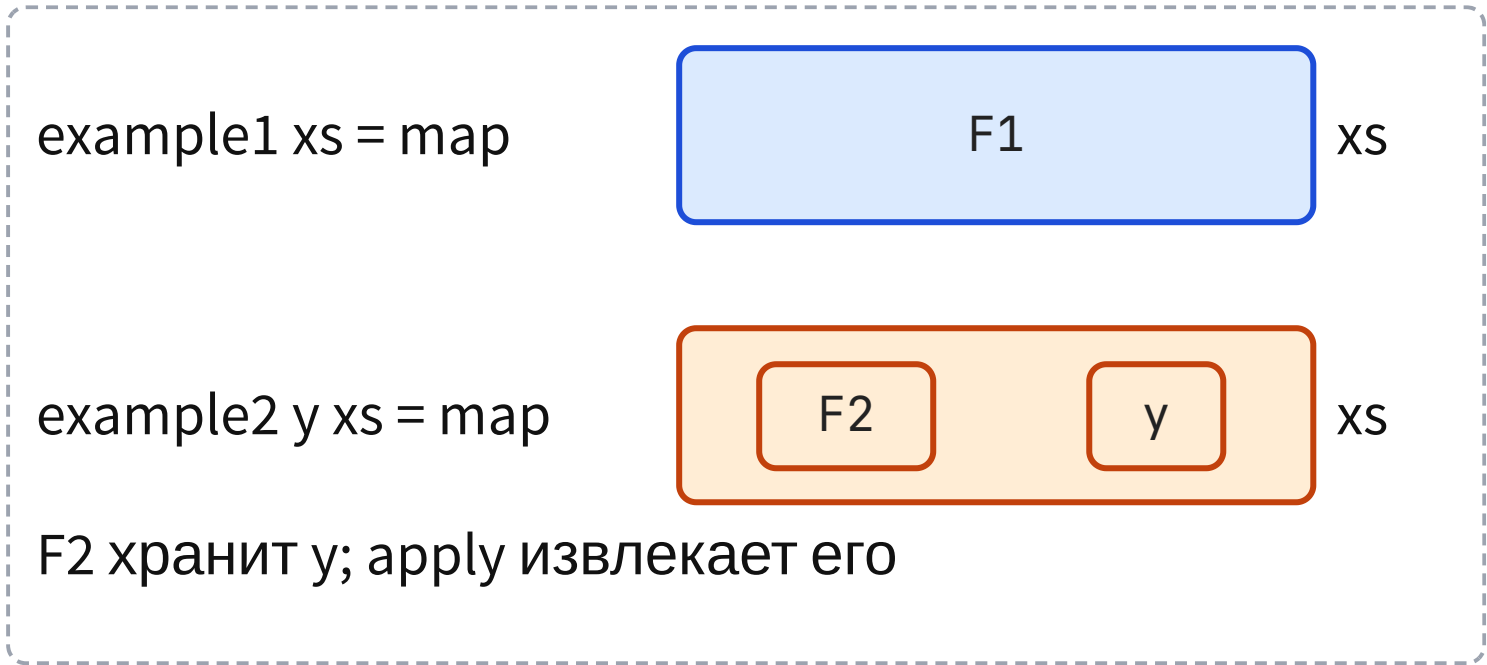


Дефункционализация: лямбды становятся данными

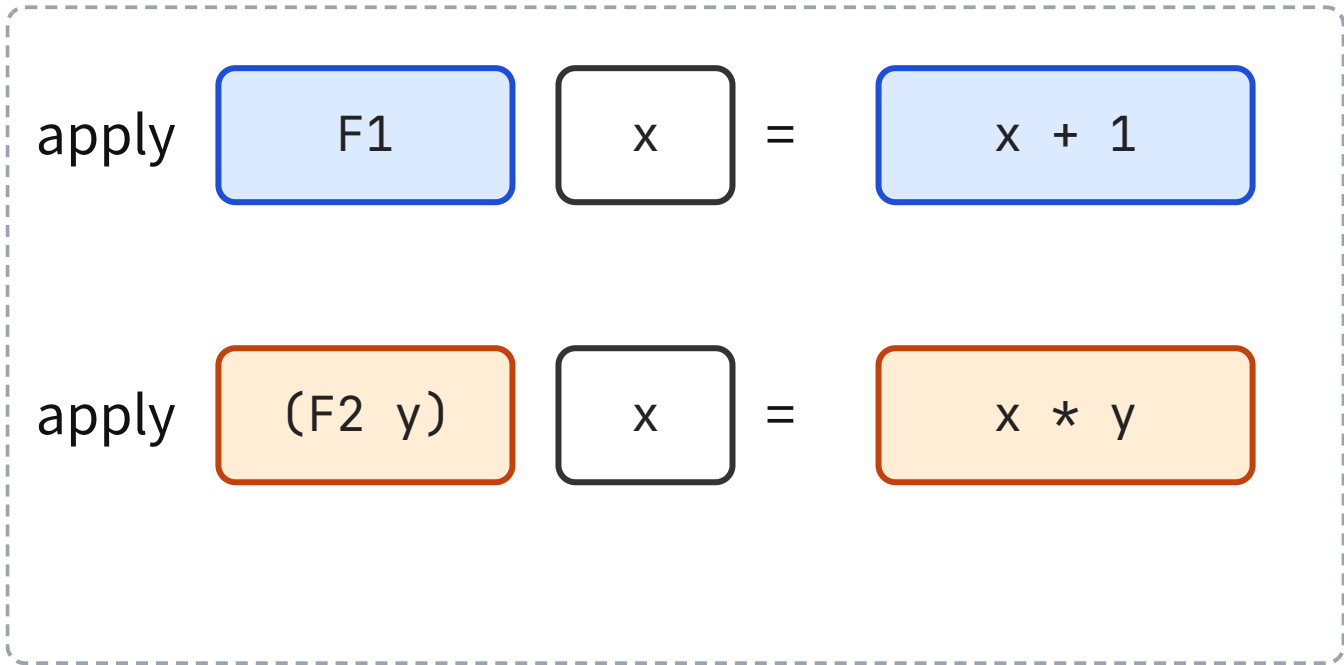
`map` теперь получает данные типа `Fun`. Вместо вызова `f x` он вызывает интерпретатор: `apply f x`.

`map :: Fun → [Int] → [Int]`

Места вызова: здесь остаются данные



`apply`: сюда переезжает код



`data Fun = F1 | F2 Int`

Вызов внутри `map`



Задача: функция приходит снаружи

```
map :: Fun -> [Int] -> [Int]
map df = \case [] -> []; x:xs -> apply df x : map df xs
```

✎ Добавим третий колсайт, в котором функция приходит снаружи: `example3 f xs = map (\x -> f x + 1) xs`. Как его дефункционализировать? Что для этого нужно знать об остальной программе?

⚡Решение.

```
data Fun = F1 | F2 Int | F3 Fun                -- f уже дефункционализована: она тоже Fun
apply (F3 f) x = apply f x + 1
```

Нужно знать все места, где создаются функции этого типа: преобразование применяется к программе целиком.

Конструктор типа становится символом

В Kotlin параметр типа не может быть конструктором типов: $M<T>$ записать нельзя.

```
interface Monad<M> { fun <T> pure(x: T):  $M<T>$  }
```


Конструктор типа становится символом

Дефункционализация на уровне типов: конструктору — тип-символ, применению — тип `Apply`.

```
interface Monad<M> { fun <T> pure(x: T): Apply<M, T> }
```

```
List<T>  $\cong$  Apply<ListSym, T>
```

Монада на Kotlin

```
class ListSym                                     // символ конструктора List
class Apply<Sym, T>(val value: Any)               // «Sym, применённый к T»: тип значения стёрт

fun <T> List<T>.to(): Apply<ListSym, T> = Apply(this)
fun <T> Apply<ListSym, T>.from(): List<T> = this.value as List<T>
```

```
interface Monad<M> {
    fun <T> pure(x: T): Apply<M, T>
    infix fun <T, R> Apply<M, T>.bind(k: (T) -> Apply<M, R>): Apply<M, R>
}

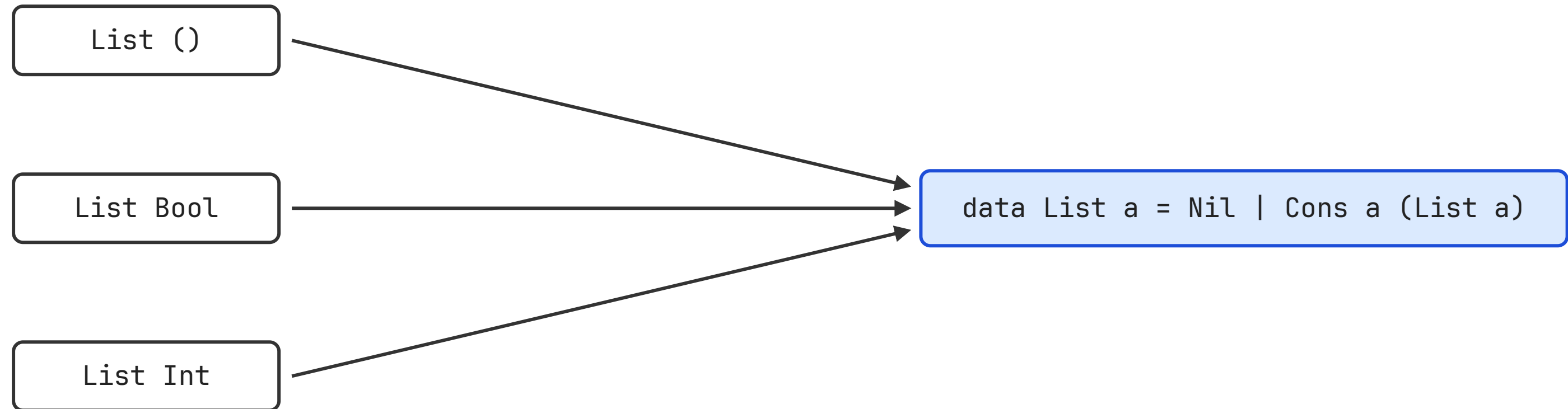
fun <M> Monad<M>.go(x: Apply<M, Int>): Apply<M, Int> =
    x bind { pure(it + 1) } bind { pure(it + 2) }
```

- — Цена: код выглядит чудовищно; скорее всего, техника себя не окупает

Семейства

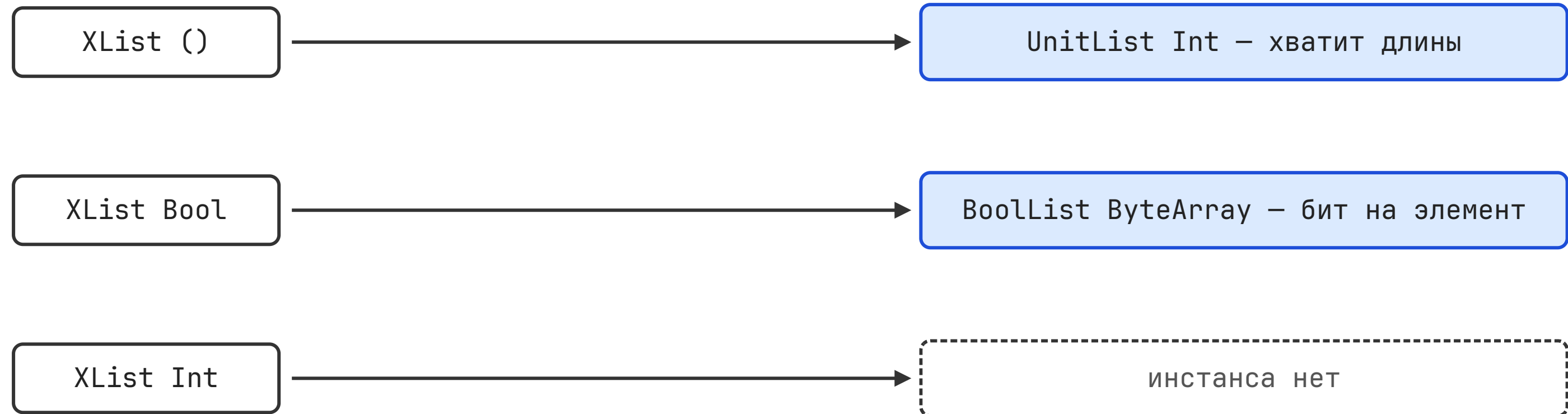
Тип выбирает представление

Обычный конструктор типа параметричен: одно объявление на все аргументы.



Тип выбирает представление

Семейство выбирает объявление по аргументу-индексу — как класс типов выбирает код.



Семейство данных живёт вместе с классом

```
data family XList elem
data instance XList ()    = UnitList Int
data instance XList Bool = BoolList ByteArray
```

```
class XListOp elem where
  xelem :: elem -> XList elem -> Bool

instance XListOp () where
  xelem () (UnitList size) = size > 0

xelemAll :: XListOp elem => XList elem -> [elem] -> Bool
xelemAll xs = all (`xelem` xs)
```

Семейства типов — функции на типах

```
type family Plus (n :: Nat) (m :: Nat) :: Nat where      -- закрытое: уравнения по порядку
  Plus Zero m = m
  Plus (Suc n) m = Suc (Plus n m)
```

```
ghci> :k! Plus (Suc Zero) (Suc (Suc Zero))
Plus (Suc Zero) (Suc (Suc Zero)) :: Nat
= Suc (Suc (Suc Zero))
```

Plus вычисляется

Подходит второе уравнение: $\text{Plus } (\text{Suc } n) \ m = \text{Suc } (\text{Plus } n \ m)$.

$\text{Plus } (\text{Suc } \text{Zero}) (\text{Suc } (\text{Suc } \text{Zero}))$

Plus вычисляется

Теперь первое: $\text{Plus Zero } m = m$.

Suc (Plus Zero (Suc (Suc Zero)))

Plus вычисляется

Нормальная форма. Вычисление прошло в компиляторе — так же в главе 2 редуцировался синоним **Pair**.

Suc (Suc (Suc Zero))

Задача: длина списка типов

✎ Реализуйте закрытое семейство `Length :: [Type] -> Nat`, вычисляющее длину списка типов.

⚡Решение.

```
type family Length (tys :: [Type]) :: Nat where
  Length '[] = Zero
  Length (ty : tys) = Suc (Length tys)
```

```
ghci> :k! Length '[Int, Bool, Char]
= Suc (Suc (Suc Zero))
```

Ассоциированные семейства

```
class Container c where
  type Elem c                -- тип, который класс ставит в соответствие c
  elements :: c -> [Elem c]

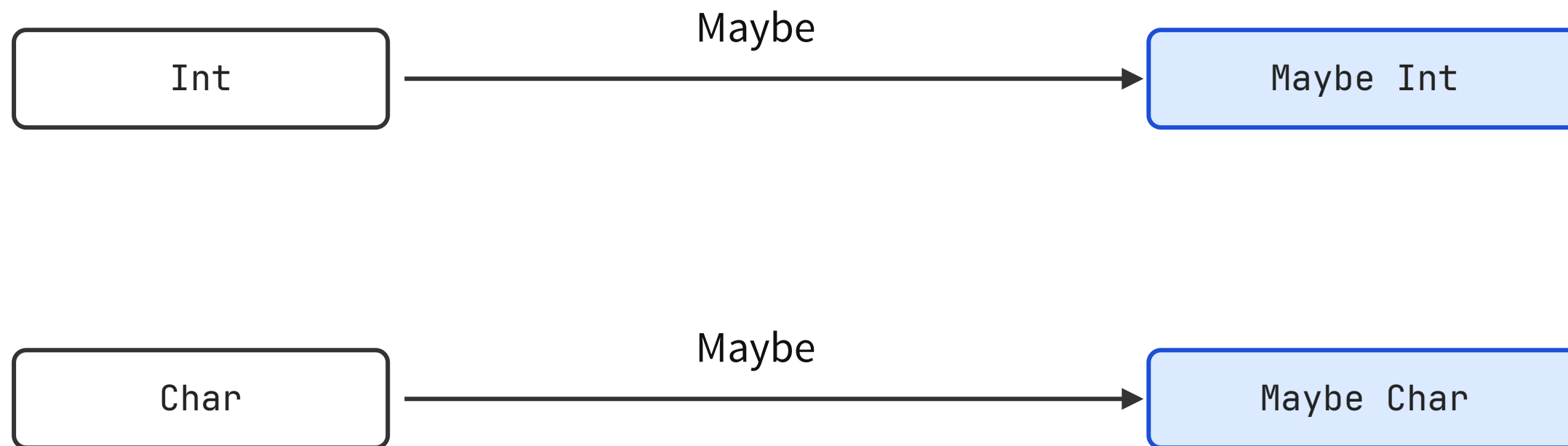
instance Container [a] where
  type Elem [a] = a
  elements = id

instance Container ByteString where
  type Elem ByteString = Word8
  elements = ByteString.unpack
```

```
ghci> :k! Elem ByteString
= Word8
```

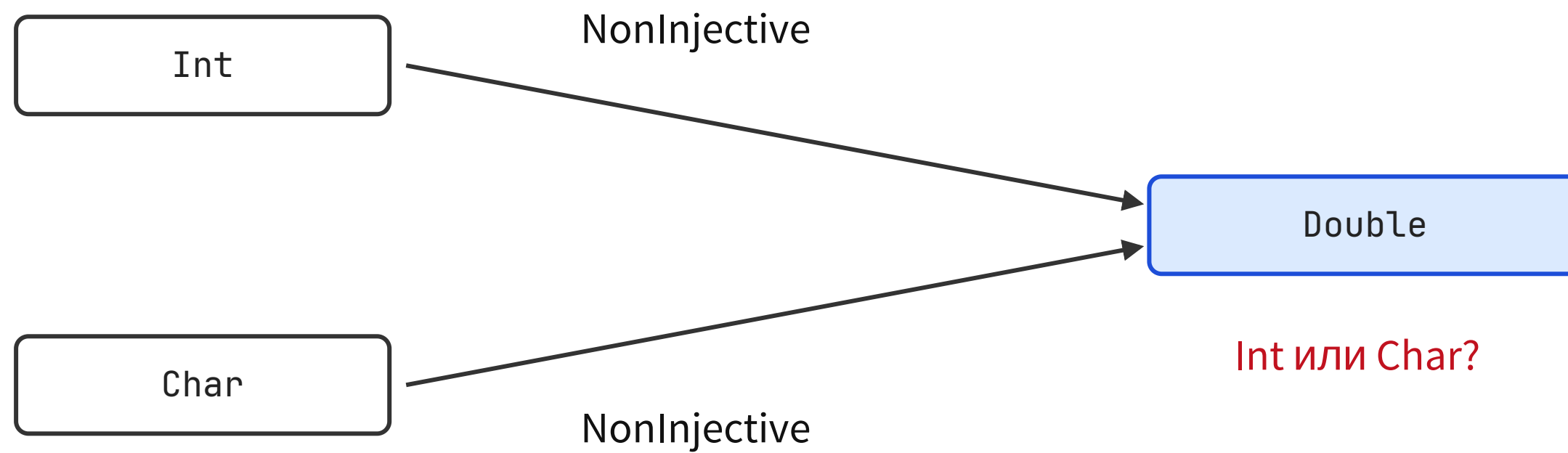
Можно ли пройти назад

Конструктор типа пассивен: разные аргументы дают разные типы. Из $\text{Maybe } a \sim \text{Maybe } b$ следует $a \sim b$.



Можно ли пройти назад

Семейство вычисляется, и разные аргументы могут дать один результат. По результату аргумент не восстановить.



Инъективность объявляют явно

```
f :: NonInjective a -> String
```

```
error: Couldn't match type: NonInjective a0
      with: NonInjective a
      NB: 'NonInjective' is a non-injective type family
      The type variable 'a0' is ambiguous
```

```
type family Inj a = r | r -> a where      -- TypeFamilyDependencies: результат определяет аргумент
  Inj Int = Bool
  Inj Char = Double
```

Семейство нельзя передать аргументом

```
type family Map (f :: Type -> Type) (as :: [Type]) :: [Type] where
  Map f '[] = '[]
  Map f (a : as) = f a : Map f as

type family Id (a :: Type) :: Type where
  Id a = a

type Test = Map Id '[Int, Bool]

error: The type family 'Id' should have 1 argument, but has been given none
```

- **Δ Matchability** — инъективность вместе с generativity: из $f\ a \sim g\ a$ следует $f \sim g$. Вывод типов предполагает её у всякого типа стрелочного кайнда

Обход: снова дефункционализация

```
type family Apply (sym :: Type) (a :: Type) :: Type      -- открытое семейство-интерпретатор
data IdSym                                                -- символ вместо семейства Id
type instance Apply IdSym a = a

type family Map (f :: Type) (as :: [Type]) :: [Type] where
  Map f '[] = '[]
  Map f (a : as) = Apply f a : Map f as
```

```
ghci> :k! Map IdSym '[Int, Bool]
= [Int, Bool]
```

Кайнд Constraint

У ограничений есть кайнд

```
ghci> :k Monad
Monad :: (Type -> Type) -> Constraint      -- класс типов строит ограничение

ghci> :k Int ~ Bool
Int ~ Bool :: Constraint                   -- равенство типов – ограничение

ghci> :k (Eq Int, Int ~ Bool)
(Eq Int, Int ~ Bool) :: Constraint         -- кортеж ограничений – ограничение
```

```
data Dict (c :: Constraint) where          -- словарь как объект языка
  Dict :: c => Dict c
```

Ограничение на каждый тип из списка

```
data HList (tys :: [Type]) where                                -- гетерогенный список из главы 2
  HNil :: HList '[]
  HCons :: ty -> HList tys -> HList (ty : tys)
```

```
type family All (c :: k -> Constraint) (tys :: [k]) :: Constraint where
  All c '[] = ()
  All c (ty : tys) = (c ty, All c tys)
```

All: семейство строит ограничение

Семейство возвращает не тип, а ограничение. Второе уравнение отщепляет первый тип списка.

All Show '[Int, Double]

```
type family All (c :: k -> Constraint) (tys :: [k]) :: Constraint where
  All c '[] = ()
  All c (ty : tys) = (c ty, All c tys)
```


All: семейство строит ограничение

Ещё раз второе уравнение.

(Show Int, All Show '[Double])

```
type family All (c :: k -> Constraint) (tys :: [k]) :: Constraint where
  All c '[] = ()
  All c (ty : tys) = (c ty, All c tys)
```

All: семейство строит ограничение

Список пуст — первое уравнение, пустое ограничение.

(Show Int, (Show Double, All Show '[]))

```
type family All (c :: k -> Constraint) (tys :: [k]) :: Constraint where
  All c '[] = ()
  All c (ty : tys) = (c ty, All c tys)
```

All: семейство строит ограничение

Получился вложенный кортеж ограничений — по одному на элемент списка.

(Show Int, (Show Double, ()))

```
type family All (c :: k -> Constraint) (tys :: [k]) :: Constraint where
  All c '[] = ()
  All c (ty : tys) = (c ty, All c tys)
```


map по гетерогенному списку

```
hmap :: forall c res tys . All c tys
      => (forall ty . c ty => ty -> res) -> HList tys -> [res]
hmap f = \case
  HNil -> []
  HCons x xs -> f x : hmap @c f xs
```

```
ghci> hmap @Show show (HCons (1 :: Int) $ HCons 'a' HNil)
["1", "'a'"]
```

Задача: сравнение гетерогенных списков

✎ Реализуйте сравнение гетерогенных списков `heq :: All Eq tys => HList tys -> HList tys -> Bool`. Откуда в рекурсивном вызове берётся словарь для хвоста списка?

⚡Решение.

```
heq :: All Eq tys => HList tys -> HList tys -> Bool
heq HNil HNil = True
heq (HCons x xs) (HCons y ys) = x == y && heq xs ys
```

Сопоставление с `HCons` даёт `tys ~ ty : tys'`, и `All Eq tys` вычисляется в пару `(Eq ty, All Eq tys')`: первая компонента нужна для `==`, вторая — словарь для хвоста.

Ограничения с кванторами

```
data Rose f x = Rose x (f (Rose f x))

instance (Eq a, forall b. Eq b => Eq (f b)) => Eq (Rose f a) where      -- QuantifiedConstraints
    Rose x1 rs1 == Rose x2 rs2 = x1 == x2 && rs1 == rs2
```

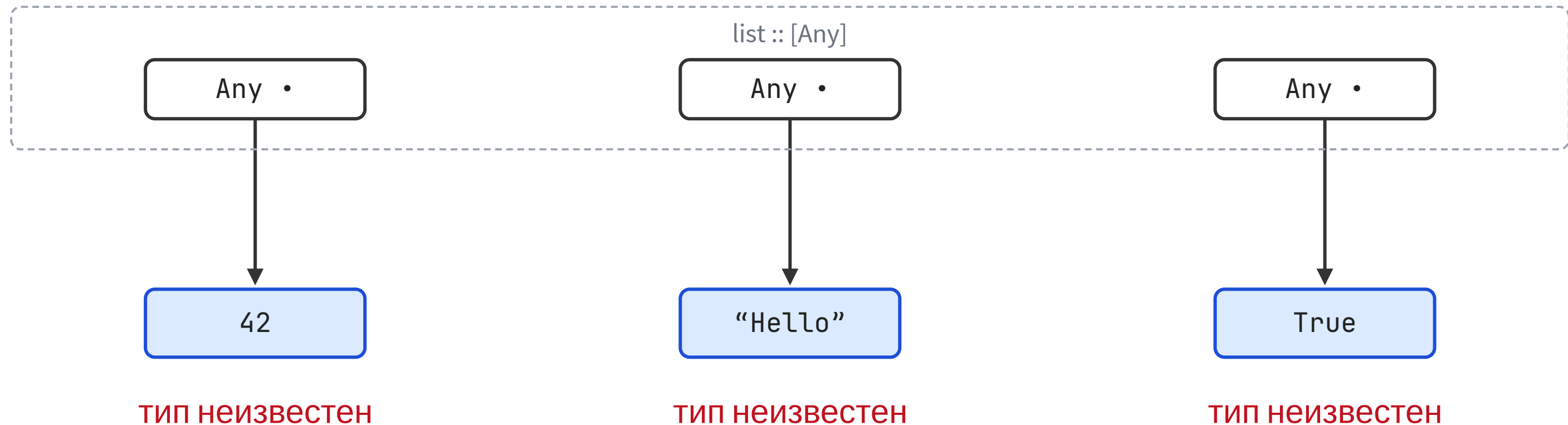
Использование ad-hoc полиморфизма

Сериализатор пары — это `listOrd`

```
interface KSerializer<T> {  
    fun serialize(encoder: Encoder, value: T)  
    fun deserialize(decoder: Decoder): T  
}  
  
class PairSerializer<K, V>(  
    keySerializer: KSerializer<K>, // объекта ещё нет: методом объекта не сделать  
    valueSerializer: KSerializer<V>, // словари компонент передаются руками  
) : KSerializer<Pair<K, V>> { ... }
```

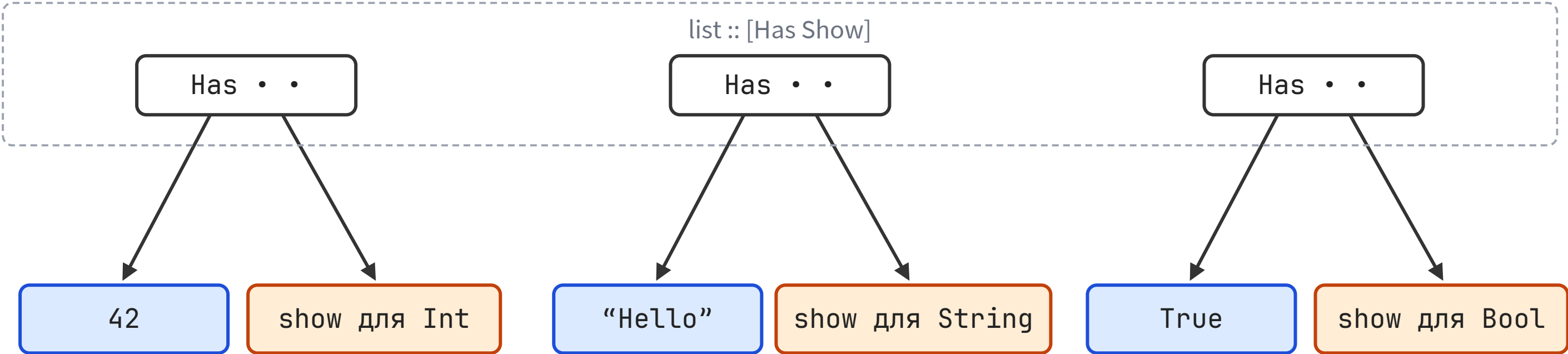

Спрятать тип, оставить словарь

Конструктор `Any :: a -> Any` прячет тип значения. В один список попадают значения разных типов — но сделать с ними нельзя ничего.



Спрятать тип, оставить словарь

Положим рядом со значением словарь — свидетельство того, что скрытый тип принадлежит классу.



Экзистенциальный тип с ограничением

```
data Has (c :: Type -> Constraint) where
  Has :: c a => a -> Has c           -- тип a скрытан, словарь с a сохранён

showAll :: [Has Show] -> String
showAll = List.intercalate ", " . map \(Has x) -> show x
```

```
ghci> showAll [Has (42 :: Int), Has "Hello", Has (Just True)]
"42,\"Hello\",Just True"
```

```
foldHas :: Has c -> (forall a . c a => a -> b) -> b           -- общий способ разобрать Has
foldHas (Has x) k = k x
```


Задача: достать значение из обёртки

✎ **Устно:** Почему нельзя написать функцию `unHas :: Has c -> a`, достающую значение из обёртки? Чем от неё отличается `foldHas`?

⚡ Решение.

```
unHas (Has x) = x
```

```
error: Could not deduce 'a1 ~ a' from the context: c a1
      'a1' is a rigid type variable bound by a pattern with constructor: Has
```

В `unHas` тип `a` выбирает вызывающий, а в обёртке лежит значение типа, выбранного при её создании. В `foldHas` квантор стоит слева от стрелки: продолжение обязано принять любой тип, и скрытый тип не покидает своей области — как `s` в `runST`.

Имя поля выбирается по типу

```
class IsLabel (s :: Symbol) a where
  fromLabel :: a                                -- OverloadedLabels: #name ≡ fromLabel @"name"

data Pet = Pet { name :: String }
instance (r ~ String) => IsLabel "name" (Pet -> r) where
  fromLabel Pet{ name } = name

data Person = Person { name :: String, pets :: [Pet] }
instance (r ~ String) => IsLabel "name" (Person -> r) where
  fromLabel Person{ name } = name
```

```
ghci> #name pet
"Rex"
```

Constraint trick: сначала голова, потом посылки

Инстанс для $Pet \rightarrow String$. Тип результата $\#name \text{ pet}$ пока неизвестен, а цель сравнивается только с головой инстанса: не совпало.

цель: $IsLabel \text{ "name" } (Pet \rightarrow r_0)$

голова: $IsLabel \text{ "name" } (Pet \rightarrow String)$

Constraint trick: сначала голова, потом посылки

Оставим голову общей — она подойдёт к любой цели. Равенство стоит в посылке и сработает уже после выбора инстанса.

цель: IsLabel "name" (Pet $\rightarrow$ r_0)

голова: IsLabel "name" (Pet $\rightarrow$ r)

посылка: $r \sim \text{String}$

Равенство типов — тоже ограничение

```
f :: forall a b . a ~ b => a -> b      -- четыре параметра: два типа, коерция и аргумент  
f = id
```

- **Δ Коерция** — свидетель равенства двух типов: значение размера 0, которое строит компилятор
- **Δ System FC** — System F с несинтаксическими равенствами типов: маленький типизированный язык, в который транслируется весь Haskell

GADT — это равенства типов

Конструкторы GADT из главы 2: у каждого свой тип результата.

$\text{Const} :: \text{Int} \rightarrow \text{Expr } \text{Int}$

$\text{IsZero} :: \text{Expr } \text{Int} \rightarrow \text{Expr } \text{Bool}$

GADT — это равенства типов

В System FC результат у всех один, а уточнение записано равенством. Сопоставление с образцом вносит это равенство в ветку.

$\text{Const} :: \forall \text{ty}. \text{ty} \sim \text{Int} \Rightarrow \text{Int} \rightarrow \text{Expr ty}$

$\text{IsZero} :: \forall \text{ty}. \text{ty} \sim \text{Bool} \Rightarrow \text{Expr Int} \rightarrow \text{Expr ty}$

Семейства — это аксиомы

```
type family Plus (n :: Nat) (m :: Nat) :: Nat where
  Plus Zero m = m
  Plus (Suc n) m = Suc (Plus n m)
```

```
axiom Plus Zero m ~ m -- порождает компилятор
axiom Plus (Suc n) m ~ Suc (Plus n m)
```


Задача: вектор в равенствах

✎ Рассахарьте в равенства типов вектор `Vec` из главы 2. Какие равенства окажутся в области видимости в каждой из веток функции `vzip`?

⚡Решение.

```
data Vec n a where
  VNil  :: forall n a . n ~ Zero => Vec n a
  VCons :: forall n a m . n ~ Suc m => a -> Vec m a -> Vec n a
```

В первой ветке `vzip` — `n ~ Zero`. Во второй — `n ~ Suc m` от первого аргумента и `n ~ Suc m'` от второго; отсюда `m ~ m'`, потому что `Suc` — конструктор, и рекурсивный вызов типизируется.

Обход, который ничего не делает

```
newtype Csv = Csv { unCsv :: String }

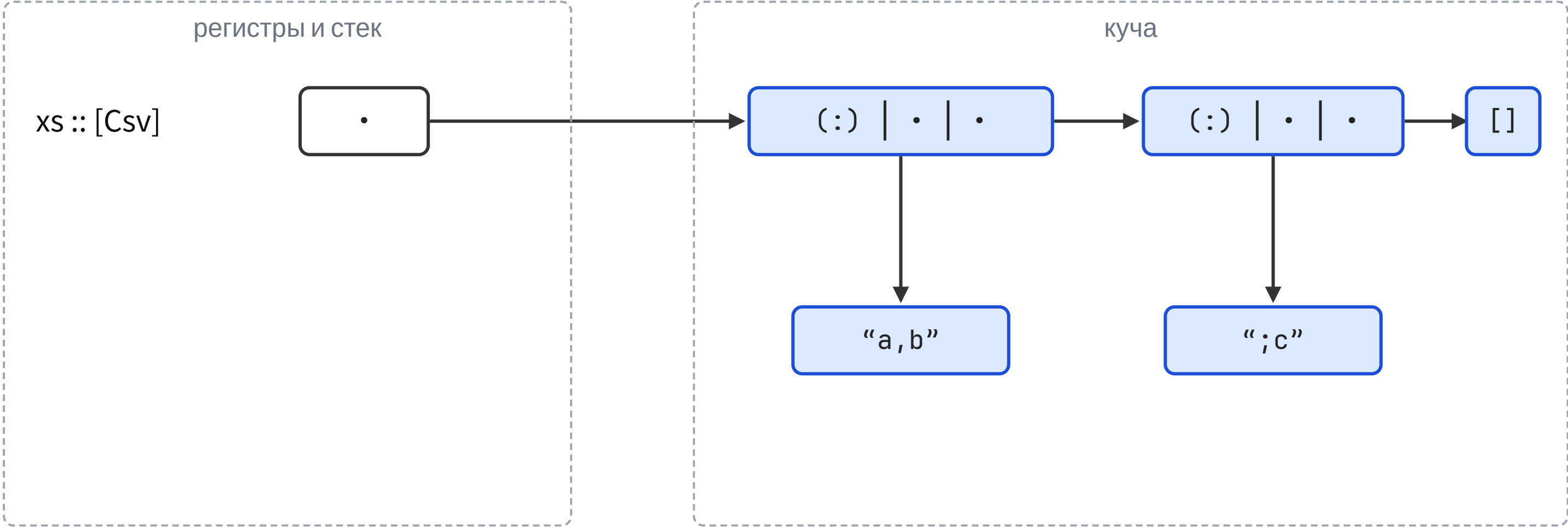
concatC :: [Csv] -> Csv
concatC = Csv . concat . map unCsv           -- map проходит по списку и ничего не меняет
```

```
concatC :: [Csv] -> Csv
concatC = coerce (concat :: [String] -> String)

class Coercible from to where
    coerce :: from -> to           -- упрощённо; инстансы строит компилятор
```

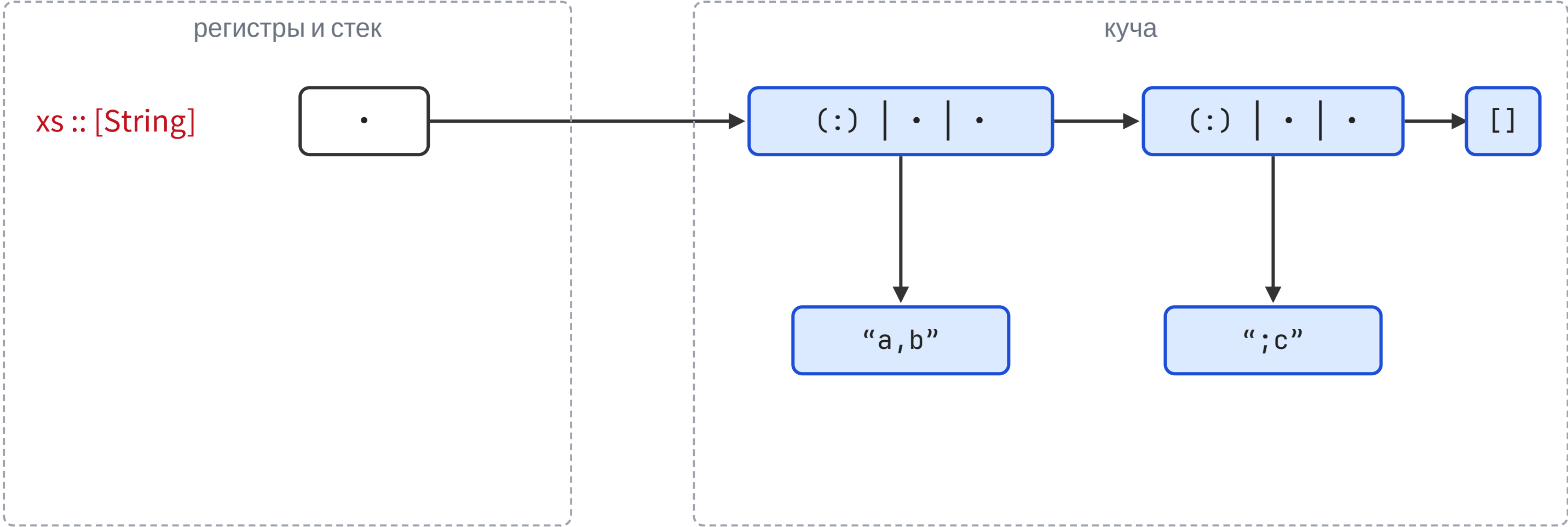
coerce меняет тип, а не память

Список обёрток в куче. Конструктора `Csv` во время исполнения нет.



coerce меняет тип, а не память

После `coerce` изменился только тип. Ни одна ячейка не тронута, обхода нет.



Роли: когда приводить можно

Роль параметра	Пример	$T\ a$ приводится к $T\ b$, если
phantom	<code>data Phantom a = Phantom</code>	всегда
representational	<code>data Box a = Box a</code>	a приводится к b
nominal	<code>data App a = App (F a)</code>	$a \sim b$

```
{-# LANGUAGE RoleAnnotations #-}
type role Map nominal representational
data Map k v = Tip | Bin k v (Map k v) (Map k v)
-- по одной роли на параметр: k, v
-- упрощённое дерево поиска
```

```
ghci> coerce (Just (ModuleId 1)) :: Maybe CourseId
Just (CourseId 1)

ghci> coerce m :: Map (Down Int) String
error: Couldn't match type 'Int' with 'Down Int' arising from a use of 'coerce'
-- m :: Map Int String
```


Задача: роль ключа в Map

✎ **Устно:** Почему у первого параметра `Map k v` роль `nominal`? Приведите пример обёртки над ключом, для которой приведение `Map k v` к `Map k' v` сломало бы словарь.

⚡ **Решение.**

Дерево поиска упорядочено по инстансу `Ord k`, а он зависит от имени типа, а не от представления. `Down Int` устроен в памяти как `Int`, но порядок у него обратный: после приведения дерево осталось бы прежним, а поиск пошёл бы по нему с другим порядком — как в задаче про множество с двумя порядками.

Тип во время исполнения

```
typeName :: forall a. Typeable a => String           -- инстансы Typeable строит компилятор
typeName = tyConName $ typeRepTyCon $ typeRep $ Proxy @a

ghci> typeName @(Maybe Int)
"Maybe"
```

```
data Dynamic where
  Dynamic :: Typeable a => a -> Dynamic              -- экзистенциальный тип со словарём Typeable

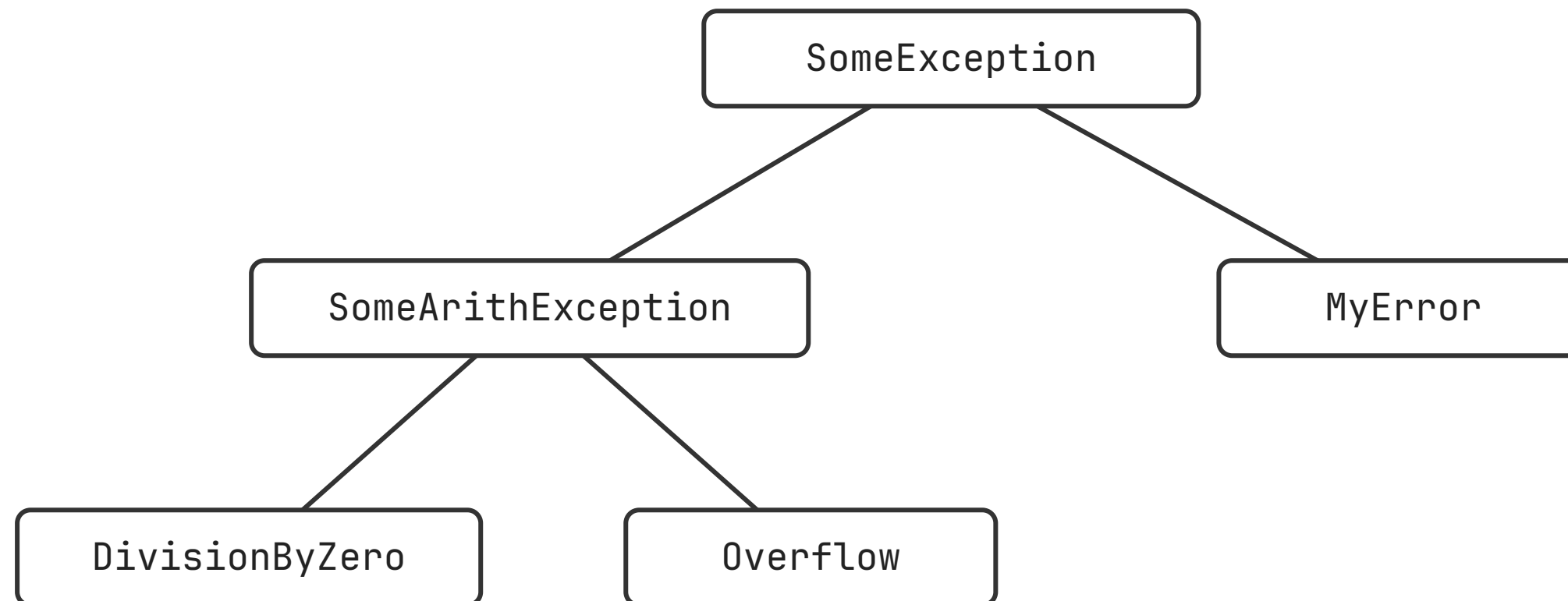
ghci> d = toDyn (42 :: Int)
ghci> fromDynamic d :: Maybe Int
Just 42
ghci> fromDynamic d :: Maybe Bool
Nothing
```

Задача: список типов в строку

✎ Реализуйте класс `ShowT (tys :: [Type])` с методом `showTypes :: String`, который позволяет распечатать список типов.

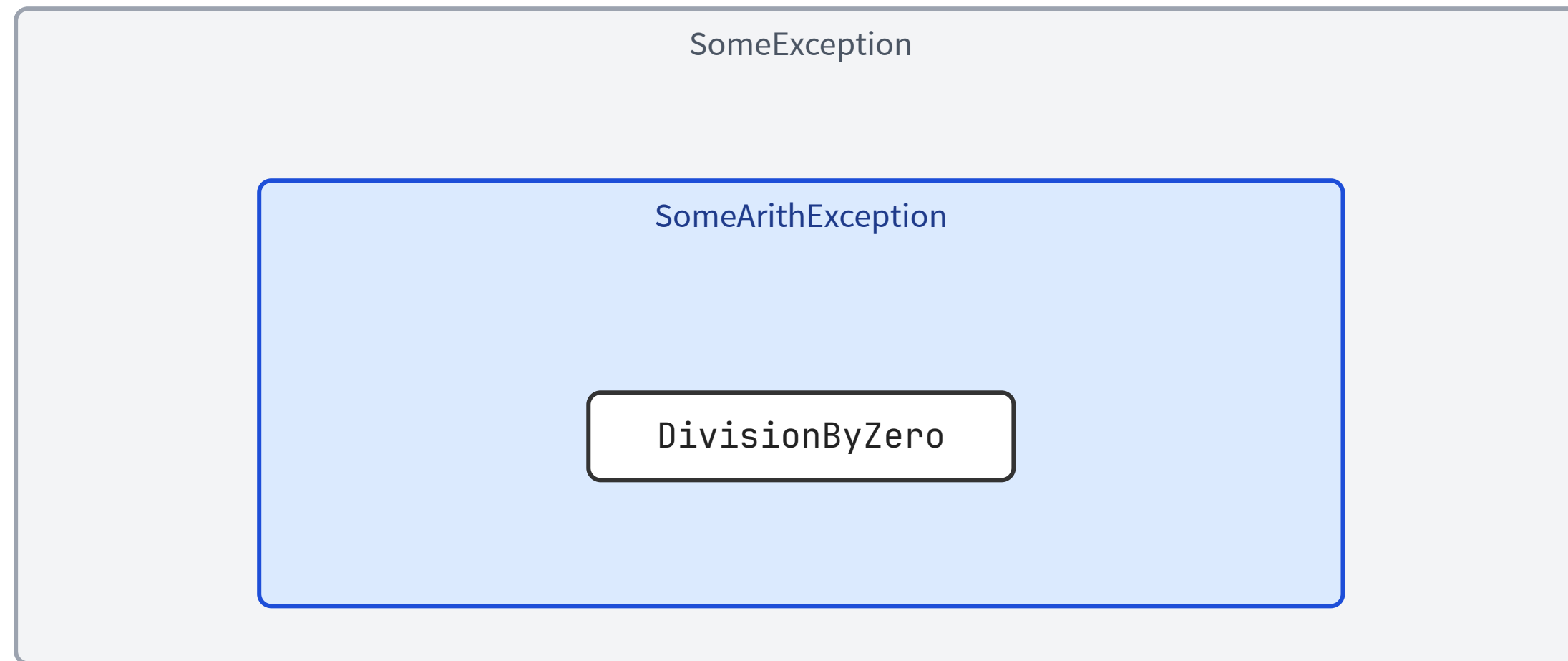
Исключение и его предки

Ошибки обрабатывают группами — нужна иерархия. Наследования в языке нет, а иерархия должна быть открытой: новый тип добавляется в любом модуле.



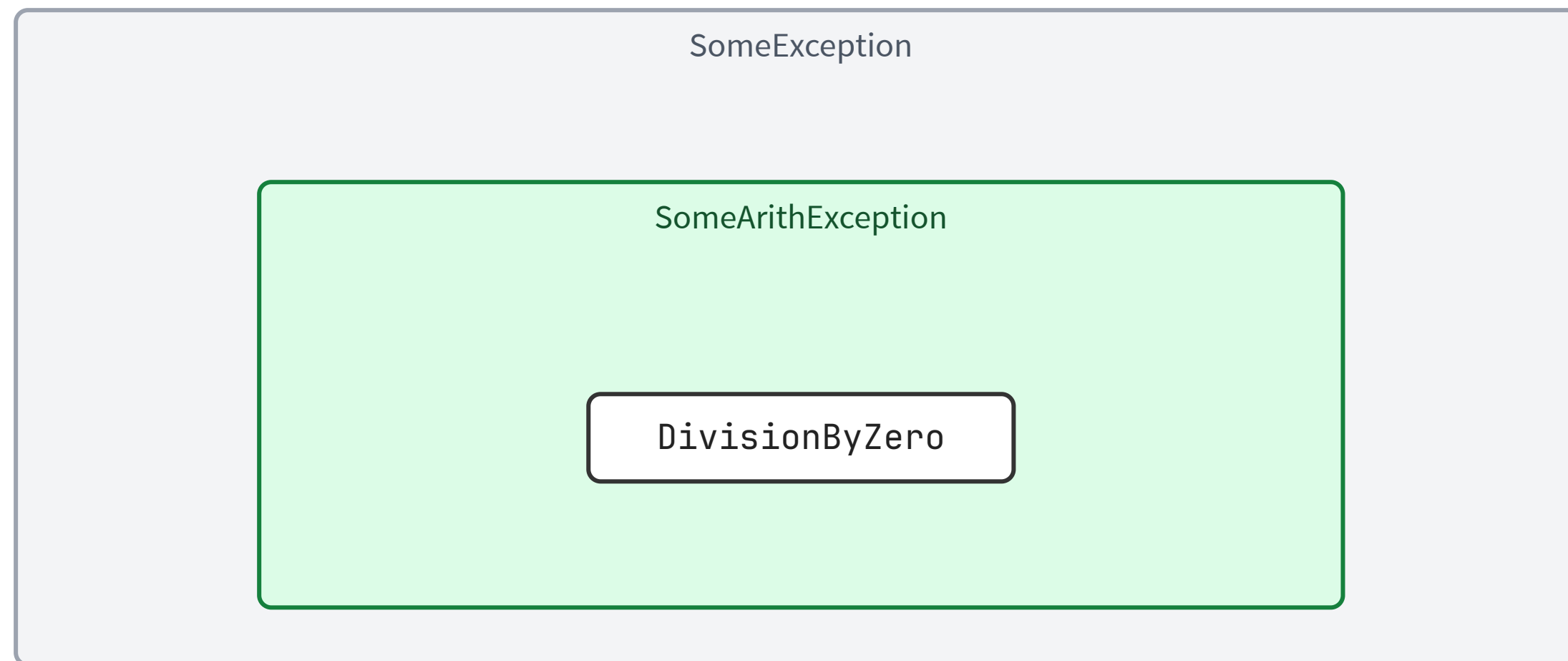
Исключение и его предки

`throw DivisionByZero`: метод `toException` заворачивает значение в обёртку всех предков.



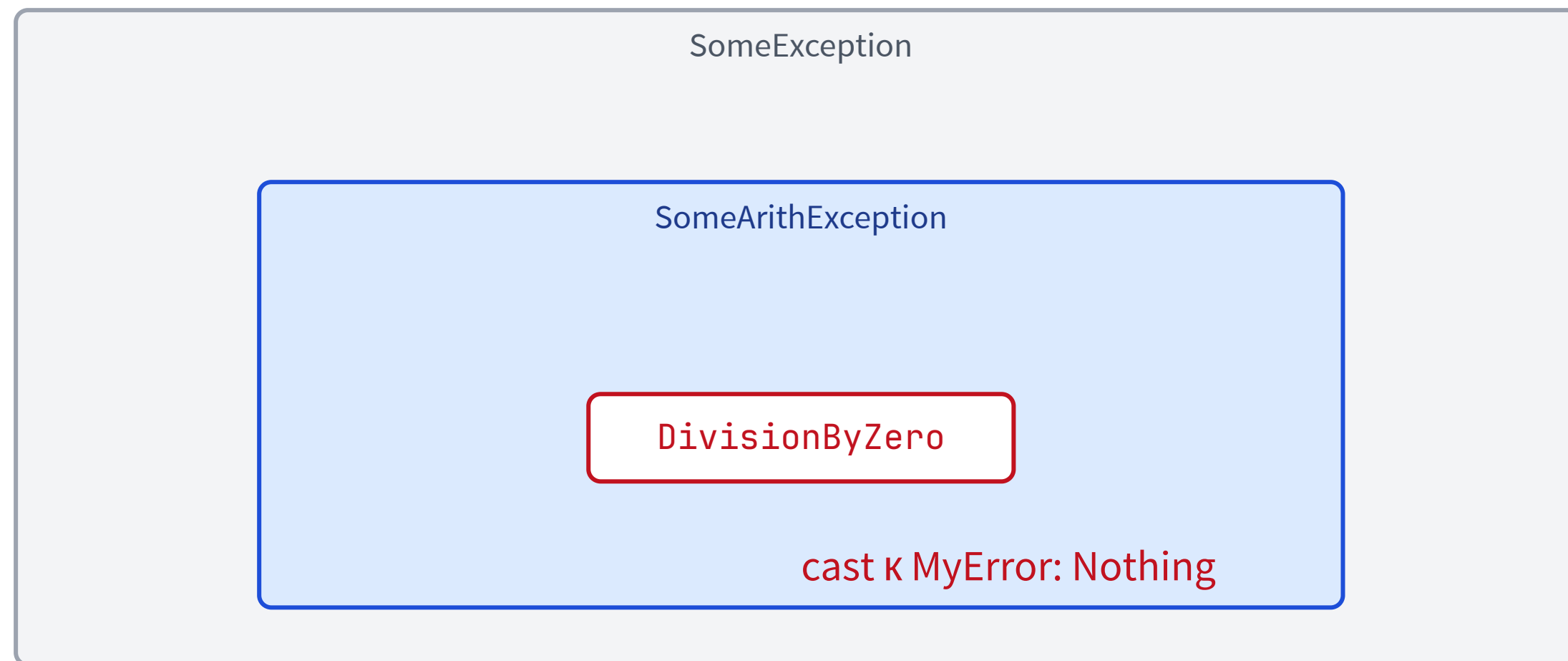
Исключение и его предки

Обработчик для `SomeArithException: fromException` снимает обёртки снаружи внутрь. Нужный конструктор нашёлся — поймали.



Исключение и его предки

Обработчик для `MyError`: обёртка снята, но внутри значение другого типа, `cast` не удался. Исключение летит дальше.



Класс Exception

```
class (Typeable a, Show a) => Exception a where
  toException :: a -> SomeException
  toException = SomeException           -- по умолчанию родитель – корень

  fromException :: SomeException -> Maybe a
  fromException (SomeException e) = cast e
```

```
data SomeException where
  SomeException :: Exception a => a -> SomeException  -- экзистенциальная обёртка, как Dynamic
```


Промежуточный уровень иерархии

```
data SomeArithException where
  SomeArithException :: Exception a => a -> SomeArithException
instance Exception SomeArithException           -- родитель – SomeException

data DivisionByZero = DivisionByZero deriving Show
instance Exception DivisionByZero where
  toException = toException . SomeArithException -- заворачиваемся в родителя
  fromException e = do
    SomeArithException e' <- fromException e      -- снимаем обёртку родителя
    cast e'
```

```
ghci> throwIO DivisionByZero `catch` \(e :: SomeArithException) -> putStrLn "caught"
caught
```

```
ghci> throwIO DivisionByZero `catch` \(e :: MyError) -> putStrLn "caught"
*** Exception: DivisionByZero
```

Задача: ещё одно арифметическое исключение

✎ Добавьте в иерархию исключение `Overflow`, тоже арифметическое. Какими обработчиками оно ловится, а какими нет?

⚡ Решение.

```
data Overflow = Overflow deriving Show
instance Exception Overflow where
  toException = toException . SomeArithException
  fromException e = do
    SomeArithException e' <- fromException e
    cast e'
```

Ловится обработчиками для `Overflow`, `SomeArithException` и `SomeException`; обработчики для `DivisionByZero` и `MyError` его пропустят.

Стек вызовов — неявный параметр

```
myHead :: HasCallStack => [a] -> a
myHead []      = error "empty"
myHead (x:_)   = x

bad :: Int
bad = myHead []           -- у bad ограничения нет: выше трейс нейдёт
```

```
ghci> bad
*** Exception: empty
CallStack (from HasCallStack):
  error, called at Bad.hs:5:17 in main:Main
  myHead, called at Bad.hs:9:7 in main:Main
```

```
type HasCallStack = (?callStack :: CallStack)
```


Свои сообщения об ошибках типов

```
instance (TypeError
  ( Text "Attempting to show a function of type "
    :<>: Text "" :<>: ShowType (a -> b) :<>: Text ""
    :$$: Text "Did you forget to apply an argument?"
  )) => Show (a -> b) where
  show = undefined           -- до исполнения дело не дойдёт
```

```
test = show not
```

```
error: Attempting to show a function of type 'Bool -> Bool'
       Did you forget to apply an argument?
```

До какого места дошло вычисление

```
ghci> xs = [1, undefined, 3] :: [Int]
```

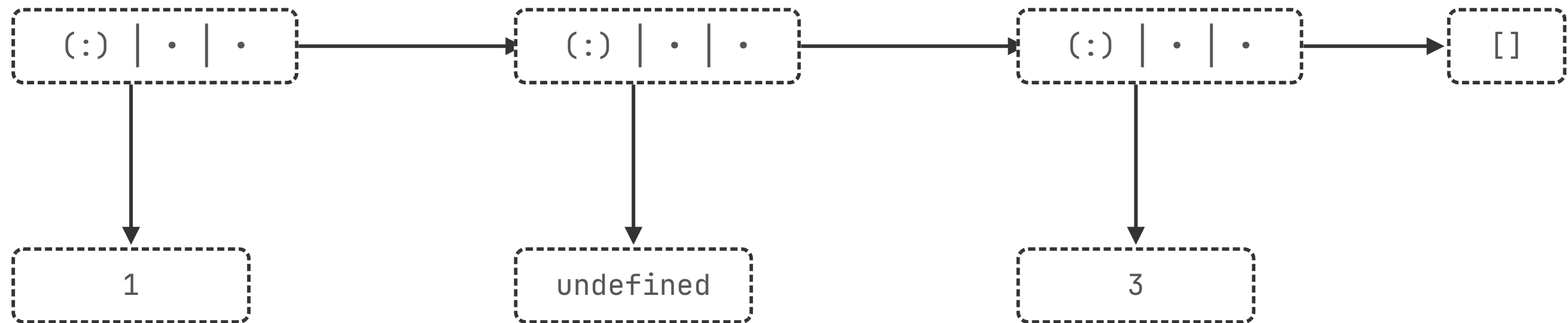
```
ghci> xs `seq` "ok"  
"ok"
```

```
ghci> length xs  
3
```

```
ghci> xs `deepseq` "ok"  
*** Exception: Prelude.undefined
```

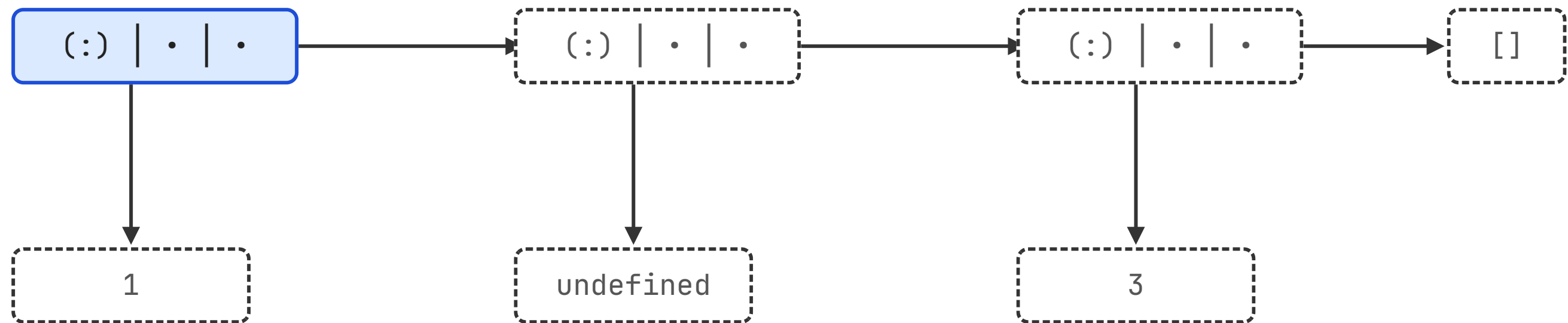
seq, length и deepseq

Пока значение никому не понадобилось, в куче лежат одни отложенные вычисления.



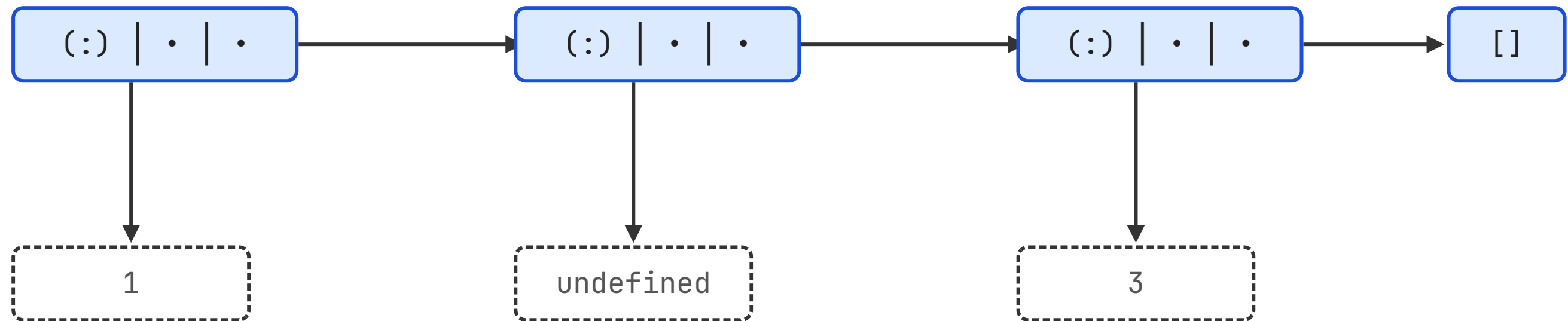
seq, length и deepseq

seq вычисляет до первого конструктора — до слабой головной нормальной формы. Поля не тронуты.



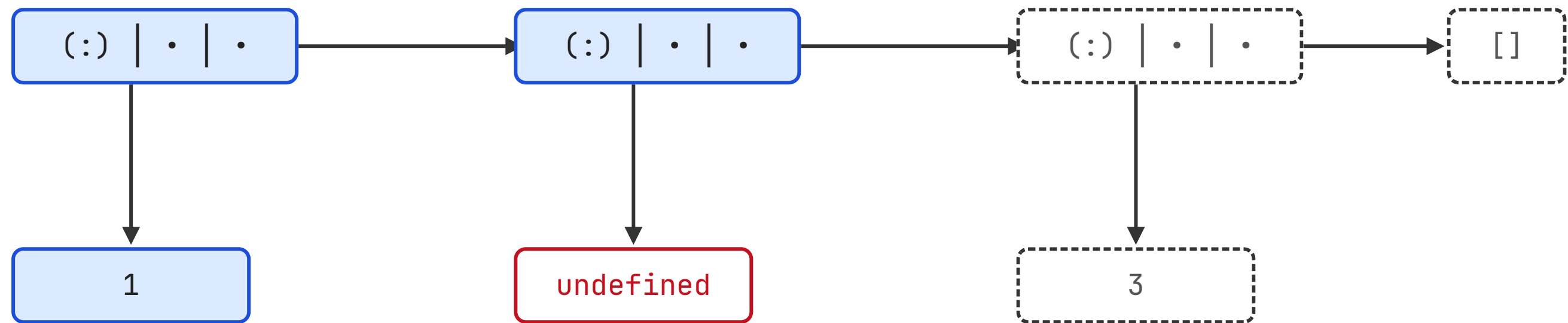
seq, length и deepseq

`length` идёт по хвостам: вычислен весь хребет списка, элементы остались отложенными.



seq, length и deepseq

`deepseq` обходит значение целиком, до нормальной формы, — и натывается на `undefined`.



Обход зависит от типа — значит, класс

```
class NFData a where
  rnf :: a -> ()                -- reduce to normal form

instance NFData Int where
  rnf x = x `seq` ()

instance NFData a => NFData [a] where
  rnf [] = ()
  rnf (x : xs) = rnf x `seq` rnf xs

force :: NFData a => a -> a
force x = rnf x `seq` x
```

```
safe, safe' :: [Int] -> IO (Either SomeException [Int])
safe xs = try (evaluate xs)          -- не поймает: вычислен только первый конструктор
safe' xs = try (evaluate (force xs)) -- поймает
```

- — Цена: `rnf` обходит значение целиком при каждом вызове, даже если оно уже вычислено

Задачи: нормальная форма

✎ Реализуйте инстанс `NFData` для дерева `data Tree a = Leaf | Node (Tree a) a (Tree a)`.

✎ **Устно:** Почему `length xs `seq` ()` не заменяет `rnf xs`? Что именно вычислит каждое из выражений?

⚡ Решение.

```
instance NFData a => NFData (Tree a) where
  rnf Leaf = ()
  rnf (Node l x r) = rnf l `seq` rnf x `seq` rnf r
```

`length xs` вычисляет хребет списка и оставляет элементы отложенными; `rnf xs` вычисляет и хребет, и каждый элемент.

Что мы узнали

классы типов	словарь, который компилятор передаёт неявно; инстанс с посылкой — функция между словарями
вывод инстансов	население типа рекурсией по его структуре; смотрит только на голову инстанса
дефункционализация	функция первого класса = данные + интерпретатор <code>apply</code> ; работает и на типах
семейства	выбор по типу для данных и типов; не инъективны и не передаются аргументами
<code>Constraint</code>	ограничения хранят (<code>Dict</code>), вычисляют (<code>All</code>), квантифицируют
System FC	равенства типов: во что транслируются GADT, семейства и <code>newtype</code>

- Домашка fr2-03: словари и инстансы, дефункционализация, семейства типов
- Следующая тема: типы данных — глава 4

Материалы

Факультативно: data reflection

```
class Reifies ty terms | ty -> terms where
  reflect :: Proxy ty -> terms

reify :: a -> (forall fresh . Reifies fresh a => Proxy fresh -> res) -> res
```

```
newtype Wrapped tag a = Wrapped { unwrap :: a }      -- фантомный тег: сколько угодно обёрток

data ReifiedOrd a = ReifiedOrd { compare :: a -> a -> Ordering }

instance Reifies tag (ReifiedOrd a) => Ord (Wrapped tag a) where
  compare = coerce $ compare $ reflect $ Proxy @tag
```

Факультативно: локальный порядок для сортировки

```
sort :: Ord a => [a] -> [a]

sortReverse :: forall a . Ord a => [a] -> [a]
sortReverse xs =
  let dict = ReifiedOrd { compare = flip compare } in
  reify dict \(Proxy :: Proxy fresh) ->
    coerce $ sort @(Wrapped fresh a) $ coerce xs
```

```
(Int, Double)    заменяем на    (Member Int d, Member Double d) => Prod d    -- открытые структуры
```

Что почитать

- Конспект курса, глава 3 «Специальный (ad-hoc) полиморфизм»
- Philip Wadler, Stephen Blott, [How to make ad-hoc polymorphism less ad hoc](#) (1989) — must-read: словари и перевод классов типов, вся первая секция в одной статье
- Oleg Kiselyov, Simon Peyton Jones, Chung-chieh Shan, [Fun with type functions](#) (2010) — must-read: семейства типов на примерах
- Martin Sulzmann et al., [System F with type equality coercions](#) (2007) — во что транслируется Haskell
- Joachim Breitner et al., [Safe zero-cost coercions for Haskell](#) (2014) — [Coercible](#) и роли
- Simon Marlow, [An extensible dynamically-typed hierarchy of exceptions](#) (2006) — открытая иерархия исключений
- Edward Kmett, [Type Classes vs. the World](#) — зачем нужна когерентность
- Chris Done, [The constraint trick for instances](#) — приём из разрешения имён и ошибок типов
- Alexis King, [Parse, don't validate](#) — практика, ради которой нужны [newtype](#) и бесплатные коерции